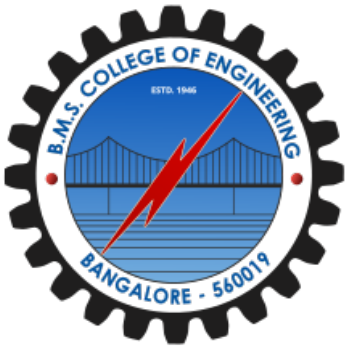
B.M.S. COLLEGE OF ENGINEERING

(Autonomous College under VTU)

Bull Temple Road, Basavangudi, Bangalore - 560019



a Thesis on

VisionFood QAI: AI-Powered Quality Inspection for Food and Beverage Manufacturing

(Project Work – Phase 2)

in fulfillment of the requirements for the award of degree

BACHELOR OF ENGINEERING

in

Computer Science & Engineering (Data Science)

by

Shreyas Bairy K S (1BM23CD058)

Sushanth S (1BM23CD063)

Ullas N (1BM23CD067)

Under the Guidance of

Prof. Nayana N Kumar

Project Guide

Department of CSE (Data Science),
B.M.S. College of Engineering

AY 2026-2027

B.M.S. COLLEGE OF ENGINEERING

(Autonomous College under VTU)

Bull Temple Road, Basavangudi, Bangalore - 560019

Department of Computer Science and Engineering (Data Science)

CERTIFICATE

This is to certify that the project entitled "**VisionFood QAI: AI-Powered Quality Inspection for Food and Beverage Manufacturing**" is a bona fide work carried out by the following students for the course Project Work – Phase 2 (Course Code: **23DS7PWPP2**). It is certified that all corrections and suggestions indicated during the internal assessments have been incorporated in the final thesis submitted to the departmental library. The project thesis has been approved as it meets the academic requirements prescribed for the Bachelor of Engineering Degree in respect of project work.

Candidate details

SL. NO.	Student Name	USN
1	Shreyas Bairy K S	1BM23CD058
2	Sushanth S	1BM23CD063
3	Ullas N	1BM23CD067

Prof. Nayana N Kumar	**Dr. Shambhavi B R**	**Dr. Bheemsha Arya**
Project Guide	Professor & HOD	Principal

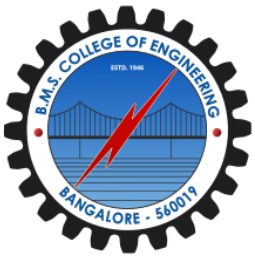
Examiners

Name of Examiner	Signature of the examiner
1.	_____
2.	_____

B.M.S. COLLEGE OF ENGINEERING

(Autonomous College under VTU)

Bull Temple Road, Basavangudi, Bangalore - 560019



DECLARATION

We hereby declare that the project thesis entitled ***"VisionFood QAI: AI-Powered Quality Inspection for Food and Beverage Manufacturing"*** is a bona fide work carried out by us under the guidance of **Prof. Nayana N Kumar**, Department of Computer Science and Engineering (Data Science), B.M.S. College of Engineering, Bengaluru, in partial fulfillment of the requirements of the degree of Bachelor of Engineering in Computer Science & Engineering (Data Science) of Visvesvaraya Technological University, Belagavi. We further declare that, to the best of our knowledge and belief, this thesis has not been submitted either in part or in full to any other university or college.

Candidate details

SL. NO.	Student Name	USN	Student's Signature
1	Shreyas Bairy K S	1BM23CD058	
2	Sushanth S	1BM23CD063	
3	Ullas N	1BM23CD067	

Date: _____ Place: Bengaluru

Acknowledgments

The satisfaction that accompanies the completion of the second phase of this ***Capstone Project*** would be incomplete without acknowledging the people whose guidance and encouragement made it possible.

We express our heartfelt gratitude to **Dr. B. S. Ragini Narayan**, *Chairperson*, Donor Trustee, Member Secretary & Chairperson, BMSET; **Dr. P. Dayananda Pai**, *Member Life Trustee*, BMSET; and **Dr. Bheemsha Arya**, *Principal*, B.M.S. College of Engineering, for providing the infrastructure required to complete this project.

We are grateful to **Dr. Shambhavi B R**, Head of the Department, Computer Science and Engineering (Data Science), and to the project coordinators for their constant support.

We express our sincere thanks to our guide, **Prof. Nayana N Kumar**, Department of Computer Science and Engineering (Data Science), for her guidance, timely reviews, and encouragement throughout both phases of the project.

We also thank all faculty members of the Department of Computer Science and Engineering (Data Science) for their views and suggestions.

Shreyas Bairy K S (1BM23CD058)
Sushanth S (1BM23CD063)
Ullas N (1BM23CD067)

Contents

Certificate i

Declaration ii

Acknowledgments iii

List of Tables 1

List of Figures 3

List of Abbreviations 5

Abstract 6

1 Introduction 7

1.1 Background . . . 7

1.1.1 Outcome of Phase 1 . . . 7

1.1.2 Direction of Phase 2 . . . 8

1.2 Objectives . . . 8

1.3 Problem Statement . . . 9

1.3.1 Scope . . . 9

1.4 Summary . . . 9

2 Literature Review 10

2.1 Visual Inspection in Food and Beverage Packaging . . . 10

2.2 Real-Time Detection and Instance Segmentation . . . 11

2.3 Knowledge Distillation . . . 11

2.4 Reduced-Precision and Accelerated Inference . . . 12

2.5 Class Imbalance and Evaluation . . . 12

2.6 Anomaly Detection for Printed Labels . . . 13

2.7 Limitations of Existing Systems . . . 13

3 Methodology 14

3.1 System Architecture . . . 14

3.1.1 Design principles . . . 14

3.1.2 Architecture overview . . . 14

3.1.3 Station 1: cap inspection . . . 14

3.1.4 Station 2: fill level . . . 15

3.1.5 Label prototype (O4) . . . 16

3.1.6 Deployment architecture . . . 16
3.1.7 Data sources and repositories . . . 16
3.2 Project Planning . . . 17
3.2.1 Gantt chart . . . 17
3.2.2 Planned versus actual progress . . . 18
3.2.3 Risk management . . . 18
3.3 Software and Hardware Requirements . . . 18

4 Implementation **21**
4.1 Algorithm Implementation . . . 21
4.1.1 Notation . . . 21
4.1.2 Segmentation network . . . 21
4.1.3 Supervised training loss . . . 22
4.1.4 Teacher–student feature distillation . . . 22
4.1.5 Asymmetric multi-view quality gate . . . 24
4.1.6 Fill-level estimation . . . 25
4.1.7 Label-defect prototype . . . 27
4.2 System Integration . . . 28
4.2.1 Pipeline per inspection . . . 28
4.2.2 Services, concurrency, and GPU memory . . . 28
4.2.3 Audit trail . . . 29
4.2.4 Operator dashboard (O5) . . . 29
4.2.5 Engine export and deployment . . . 29
4.2.6 Integration challenges . . . 31
4.3 Testing & Validation . . . 31

5 Results and Discussion **32**
5.1 Experimental Setup . . . 32
5.2 Experimental Results . . . 32
5.2.1 Cap dataset and artefacts (O1) . . . 32
5.2.2 Distillation outcome . . . 33
5.2.3 Cap segmentation: trained versus deployed model . . . 34
5.2.4 Product-level error and threshold analysis . . . 35
5.2.5 Fill-level estimation (O2) . . . 35
5.2.6 Label-defect prototype (O4) . . . 37
5.3 Performance Evaluation . . . 38
5.3.1 Network latency (O3) . . . 38
5.3.2 Closed-loop stress test . . . 38

5.4 Validation . . . 39
5.5 Comparative Analysis . . . 39
5.5.1 Why a segmentation detector replaced the Phase 1 classifier . . . 39
5.5.2 Comparison with the literature . . . 40
5.6 Model Optimization & Innovation . . . 40
5.7 Summary Against the Stated Targets . . . 41
5.8 Discussion and Limitations . . . 41

6 Conclusion and Future Discussion **43**
6.1 Conclusion . . . 43
6.2 Key Learnings . . . 43
6.3 Research Publication Status . . . 44
6.4 Future Work . . . 44
6.5 SDGs Achieved . . . 45

Appendices **46**

A Appendix A: Source Excerpts **47**

B Appendix B: Interface of the Deployed System **49**

C Appendix C: Label Prototype Calibration Record **50**

References **51**

List of Tables

2.1 Comparison of representative packaging-inspection studies . . . 11
2.2 Limitations of existing systems and the Phase 2 response . . . 13

3.1 Datasets used in Phase 2 . . . 17
3.2 Phase 1 plan versus Phase 2 outcome . . . 19
3.3 Risk register with mitigation and evidence of adjustment . . . 19
3.4 Software requirements . . . 20
3.5 Hardware requirements . . . 20

4.1 Notation . . . 21
4.2 Training configuration of the cap teacher and student . . . 24
4.3 Operator dashboard features . . . 30
4.4 Integration challenges and solutions . . . 31
4.5 Representative test cases . . . 31

5.1 Six-class cap dataset (one cap per image) . . . 32
5.2 Cap model artefacts . . . 33
5.3 Teacher versus distilled student (FP32, 640 px) . . . 33
5.4 Station 1 segmentation on the held-out test split ($n = 354$) . . . 35
5.5 Production gate replayed with the FP16 engine (defaults $\tau_p = 0.75$, $\tau_d = 0.45$, $\tau_u = 0.40$) . . . 35
5.6 Station 2 detectors on held-out test splits (640 px) . . . 36
5.7 Fill-ratio error from meniscus localisation (41 test images, TRT FP16) . . . 36
5.8 Label prototype: colour versus grayscale EfficientAD-Medium, crop-level . . . 37
5.9 Label prototype: grayscale EfficientAD-Medium per region . . . 37
5.10 Per-call latency (ms, median of 100) inside the production container . . . 38
5.11 Station 1 closed-loop stress test, 500 unpaced cycles (ms) . . . 39
5.12 Whole-image cap classifiers (separate 4-class dataset, $N_{\text{test}} = 308$). GPU: PyTorch, batch 1; CPU: ONNX Runtime INT8 . . . 39
5.13 Phase 1 versus Phase 2 cap inspection (different datasets and label sets; indicative only) . . . 40
5.14 All targets and outcomes (test split unless noted) . . . 41

6.1 SDG alignment, beneficiaries, and evidence . . . 45

B.1 REST and WebSocket endpoints . . . 49

C.1 Calibrated thresholds of the label prototype . . . 50

List of Figures

3.1 Overall VisionFood QAI architecture 15
3.2 Multi-stage fill-level pipeline (Station 2) 15
3.3 Label-defect prototype pipeline 16
3.4 Deployment architecture 17
3.5 Phase 2 Gantt chart 18

4.1 Fill-ratio geometry: trimmed bottle box (red) and meniscus mid-line (blue). . . 25
4.2 Integration flow for one Station 1 inspection 29

5.1 Student validation mAP during distillation training 34
5.2 Student precision–recall curves and confusion matrix 34
5.3 Gate threshold sweep 36

List of Abbreviations

Abbreviation	**Expansion**
AP / mAP	Average Precision / mean Average Precision
API	Application Programming Interface
C2PSA	Cross-stage partial block with Position-Sensitive Attention (YOLO11)
C3k2	Cross-stage partial bottleneck with kernel-2 option (YOLO11)
DFL	Distribution Focal Loss
FAR / FRR	False Accept Rate / False Reject Rate
FP16 / FP32	16-bit / 32-bit floating point
GFLOPs	Giga floating-point operations
IoU	Intersection over Union
KD	Knowledge Distillation
KPI	Key Performance Indicator
MCC	Matthews Correlation Coefficient
NMS	Non-Maximum Suppression
ONNX	Open Neural Network Exchange
ORT	ONNX Runtime
PET	Polyethylene terephthalate (bottle material)
REST	Representational State Transfer
ROI	Region of Interest
RTSP	Real-Time Streaming Protocol
SAHI	Slicing Aided Hyper Inference
SDG	Sustainable Development Goal
SPPF	Spatial Pyramid Pooling – Fast
TRT	NVIDIA TensorRT
VRAM	Video Random-Access Memory (GPU memory)
WAL	Write-Ahead Log (SQLite journal mode)

Abstract

Packaged food and beverage lines still depend heavily on human inspectors, who cannot keep pace with several products per second and who tire over a shift. The first phase of this project proposed a broad inspection architecture; this second phase narrowed it to inspection stations that were built, measured, and deployed. The work had six objectives: a bottle-cap defect detector with threshold analysis, an audited dataset with a defined evaluation protocol, a geometric fill-level estimator with configurable limits, accelerated inference engines, a label-defect prototype for an industrial cosmetic tube, and an operator dashboard with an audit trail. For cap inspection, the Phase 1 pipeline of a bottle detector followed by a MobileNetV3-Small classifier, which reached 86.5% accuracy, was replaced by a single YOLO11m instance-segmentation student distilled from a YOLO11x-seg teacher through normalised feature alignment at three backbone and attention modules. The student labels six cap states on three synchronised camera views, and an asymmetric quality gate converts its detections into pass, reject, review, or failed-inspection verdicts. The fill-level station combines a bottle detector, a liquid-surface detector, and a closed-form fill ratio with a review band around the underfill and overfill limits. The label prototype localises four printed regions on 1504 by 8000 pixel tube scans by template matching and scores each region with an EfficientAD anomaly model trained only on good labels. All models were exported through ONNX to TensorRT FP16 engines and served by a FastAPI backend and React dashboard in Docker, with every inspection written to SQLite. On the held-out test split of 354 images, the student matched the teacher's mask mAP at IoU 0.5 of 0.995 with 2.8 times fewer parameters and exceeded its strict-IoU mask mAP (0.927 against 0.912); the FP16 engine scored 0.928. Replaying the quality gate produced no false accepts among 153 defective caps and no false rejects among 201 good caps. The liquid-surface detector reached AP of 0.923 at IoU 0.5, the median fill-ratio error from surface localisation was 0.002, and 34 of 35 fill verdicts agreed with the annotated surface. TensorRT reduced the three-view network time from 21.0 ms to 7.4 ms; the closed loop with synchronous audit storage ran at a median of 299 ms per bottle, so storage, not inference, limits throughput. In the label prototype, template matching located all four regions on all 120 scans, and grayscale anomaly models caught 91 to 94 percent of defective crops, although one region still raised too many false alarms. The project shows that a distilled segmentation model, a transparent geometric rule, and an auditable deployment can deliver measurable inspection quality, and it contributes to SDG 9 and SDG 12 by making automated inspection affordable for small and mid-size producers.

Keywords: bottle-cap inspection; knowledge distillation; YOLO11 instance segmentation; fill-level estimation; anomaly detection; TensorRT; quality gate; operator dashboard.

Chapter 1

Introduction

This chapter places the second phase of VisionFood QAI in context. It summarises what Phase 1 delivered, explains why the scope was narrowed and the architecture changed, states the Phase 2 objectives in measurable form, and defines the problem the remaining chapters address.

1.1 Background

Visual inspection is the last line of defence between a filling or labelling machine and the consumer. On a beverage line the two most frequent visual rejects concern the closure and the fill level: a cap that is missing, open, misplaced, or damaged breaks the seal, and a container filled below or above its band misstates the declared volume and wastes product. On cosmetic and personal-care lines the printed label carries the brand, the ingredient list, the batch and expiry codes, and the barcode, so a torn, smudged, or partly missing print is a regulatory as well as a cosmetic defect.

Manual inspection does not scale to these lines. A filler running at 300 bottles per minute presents a new container every 200 ms, and inspectors examining reflective plastic or glossy print lose consistency over a shift [1]. Rule-based machine vision, built from edge detection and pattern matching, works for a fixed product but must be re-tuned whenever a bottle shape, cap colour, or label layout changes [2]. Deep convolutional networks learn the visual patterns directly from data and have reached high accuracy on food-packaging defects [1, 3], yet published systems rarely report what matters to a plant: the rate at which defective products are passed, the rate at which good products are discarded, and the end-to-end latency of the deployed system [4].

1.1.1 Outcome of Phase 1

Phase 1 surveyed eighteen studies, proposed a product-profile-driven architecture with a YOLOv8 detector, an EfficientViT-M5 confirmation classifier, Monte Carlo Dropout uncertainty, and a REMEDY routing engine, and built a first bottle prototype. The prototype detected bottles and caps with a YOLO detector (precision 0.869, recall 0.973, mAP@0.5 of 0.958, mAP@0.5:0.95 of 0.786 on a three-class cap set), classified the cap crop with MobileNetV3-Small, and computed fill ratio from a water-surface detector. The cap classifier reached 86.5% accuracy (precision and recall 0.870), and the dashboard reported 466–678 ms per uploaded image on a development machine.

The Phase 1 conference submission (CSITSS 2026, Paper 1204) was reviewed in August 2026. The reviewers asked for evaluation on an untouched test split, product-level false-accept and false-reject rates, measured latency, a justification of the cap classifier's accuracy, and a scope that matched what had actually been tested. These comments, together with the Phase 1 evaluation feedback, shaped Phase 2.

1.1.2 Direction of Phase 2

Phase 2 therefore moved from a broad architecture to a set of inspection stations that are built, measured, and deployed:

- **Station 1 (cap)** replaces the detector-plus-classifier cascade with one distilled instance-segmentation network that labels six cap states on three camera views.
- **Station 2 (fill level)** formalises the geometric fill-ratio rule, adds a review band, and measures its error.
- **Label prototype** extends inspection to printed labels on an industrial cosmetic tube supplied as the Himalaya NSC dataset.
- **Deployment** compiles every model to a TensorRT engine and serves the stations through a containerised backend, an operator dashboard, and an SQLite audit trail.

Product formats, the REMEDY engine, Monte Carlo Dropout, and SKU profiles from the Phase 1 design were not carried forward; Section 1.3.1 lists what is in and out of scope.

1.2 Objectives

Primary objective. Develop and evaluate a bottle-cap defect detector with small-object-aware localisation and threshold analysis, deployable at line rate on a single workstation GPU.
Specific objectives.

O1. Preserve and audit the cap dataset and all model artefacts (PyTorch, ONNX, TensorRT): document per-split and per-class counts, capture resolution, and object-size distribution; verify that the deployed FP16 engine preserves the accuracy of the trained model; and identify a capture-session-grouped held-out test as the benchmark for future model versions.

O2. Build a fill-level estimator from a bottle detector and a liquid-surface detector with a closed-form fill ratio and configurable underfill and overfill limits, and measure its ratio error and verdict agreement on held-out images (target: verdict agreement ≥ 0.93).

O3. Optimise inference by exporting every model to a TensorRT FP16 engine, and measure per-stage and closed-loop latency against PyTorch and ONNX Runtime baselines (targets: network time < 100 ms for three views; closed-loop cycle ≤ 200 ms for 300 bottles per minute).

O4. Prototype a label-defect detector for the Himalaya NSC tube label: template-matching localisation of four printed regions, ROI cropping, and per-region anomaly models, with honestly reported limitations.

O5. Deliver an operator dashboard with live KPIs, a review queue for uncertain products, replay of stored inspections, and an exportable SQLite audit trail, packaged with Docker.

The quantitative targets for the cap station were carried over from Phase 1: image-level accuracy ≥ 0.93, precision ≥ 0.93, recall ≥ 0.97, false-accept rate $\leq 3\%$, and false-reject rate $\leq 7\%$. All targets are compared with measured outcomes in Table 5.14, including the ones that were missed.

1.3 Problem Statement

The core problem is to decide, for every product on a moving line, whether its closure, fill level, and printed label are acceptable, with error rates and latency that a plant can verify and audit. Four difficulties make this hard.

First, the decisive evidence is small. A droplet on a wet cap, a dent on a damaged cap, the thread gap of an open cap, or a smudge on a few square millimetres of print occupies a small fraction of the region an inspector looks at. A model must keep that fine detail through its down-sampling layers.

Second, the two error types are not equally costly. Passing a defective product exposes the consumer and the brand; rejecting a good product costs material. A useful system must make the asymmetry explicit, send doubtful cases to a human, and report both error rates rather than a single accuracy figure.

Third, accuracy and speed compete. Large networks extract finer features but are too slow for a 200 ms cycle on one GPU, and a model that is fast in isolation can still miss the line rate once image transfer, post-processing, and storage are added.

Fourth, defects are scarce. Defective labels in particular are rare and varied, so a supervised classifier for each defect type cannot be trained from the few examples available.

1.3.1 Scope

In scope are transparent PET bottles (cap and fill-level stations) and the Himalaya Winter Defense Moisturizing Cream 50 ml tube label (prototype). Out of scope are other package types, chemical or microbial contamination, label checks on bottles, remediation routing, and claims about physical fill volume, because no volumetric ground truth was available.

1.4 Summary

Phase 2 turns the Phase 1 proposal into measured stations. A distilled YOLO11m segmentation model inspects caps behind an asymmetric quality gate, a geometric rule estimates fill level, an anomaly-detection prototype inspects printed labels, and all of them run as optimised engines behind an auditable dashboard. Chapter 2 reviews the literature on food and beverage inspection, real-time segmentation, knowledge distillation, anomaly detection, and inference optimisation. Chapter 3 presents the system architecture, the project plan with planned-versus-actual progress and risk management, and the requirements. Chapter 4 gives the algorithms, loss functions, and integration details. Chapter 5 reports and discusses the results against every target, and Chapter 6 concludes with learnings, future work, and the SDG alignment.

Chapter 2

Literature Review

This chapter reviews the work that the Phase 2 design builds on. Phase 1 surveyed food-quality inspection broadly; this review keeps the studies that bear directly on the stations that were built and adds the literature the Phase 2 decisions depend on: instance segmentation, knowledge distillation, reduced-precision inference, class-imbalance handling, and unsupervised anomaly detection for printed surfaces. Each theme ends with what it implies for this project.

2.1 Visual Inspection in Food and Beverage Packaging

Classical machine vision. Farhangi *et al.* [2] inspected caps, liquid level, and label placement on liquid bottles with a CMOS camera, edge detection, and pattern matching, reporting 95.6% average accuracy. The method works because the product, lighting, and camera pose are fixed; every new bottle or cap design requires new rules.

CNN classification. Banús *et al.* [1] compared ResNet, VGG, and DenseNet backbones for thermoformed food packages and validated the best model on a production line with very low false-accept and false-reject rates. Their study is one of few that report both error types, and it shows that imaging design and dataset curation matter as much as the network. Subramanian *et al.* [5] used an optimised ResNet-50 for manufacturing defects. Chatchapol *et al.* [6] classified beverage fill level and lid sealing with a CNN on an embedded GPU, reaching 98.89% liquid-level accuracy, but they classify the whole image and do not localise the liquid surface, so the result cannot be expressed as a fill ratio.

Detection. Liu *et al.* [3] added CBAM attention and adaptive feature fusion to YOLOv5 for five food-packaging defects (7,400 images, accuracy 0.96, recall 0.94). Nguyen *et al.* [7] integrated YOLOv8 with a PLC and multi-threaded acquisition and reported above 90% accuracy across defect types. Kavitha *et al.* [8] coupled YOLO detection to Arduino-driven sorting. These systems localise defects, but they report validation accuracy rather than product-level error rates, and only Nguyen *et al.* measure the latency of the full loop.

Surveys and data-centric work. Liakos *et al.* [4] reviewed machine learning for food quality control and found beverages to be a small minority of studied applications; they list data scarcity and integration with legacy systems as open problems. Xiong *et al.* [9] showed in a food-packaging case study that improving data quality can matter as much as changing the model. Sensor-heavy approaches [10, 11, 12] monitor quality along the supply chain but do not inspect visual defects on the line.

Implication. The literature supports learned localisation, but it rarely reports false-accept and false-reject rates, threshold margins, or deployed latency. Phase 2 reports all three for every station.

Table 2.1: Comparison of representative packaging-inspection studies

Study	Task	Method	Reported result	Gap relative to this project
Farhangi *et al.* [2]	Cap, level, label on bottles	Edge detection, pattern matching	95.6% accuracy	Hand-tuned rules per product
Banús *et al.* [1]	Thermoformed food packs	DenseNet, ResNet, VGG	Very low FAR/FRR on line	Classification only; no localisation
Chatchapol *et al.* [6]	Fill level, lid seal	CNN, embedded GPU	98.89% level accuracy	No surface localisation, no ratio
Liu *et al.* [3]	Five packaging defects	YOLOv5 + CBAM + ASFF	Acc. 0.96, recall 0.94	No product-level error rates or latency
Nguyen *et al.* [7]	Industrial surfaces	YOLOv8 + PLC	>90% accuracy	Not food-specific
Truong and Luong [13]	Reusable food packaging	Autoencoder anomaly detection	Detects cracks and contamination	Threshold sensitivity, false alarms
Ribeiro *et al.* [14]	Pharmaceutical bottles	Multi-view, incremental learning	Adapts to new defects	Pharmaceutical domain

2.2 Real-Time Detection and Instance Segmentation

One-stage detectors in the YOLO family predict boxes and classes in a single forward pass [15]. Ultralytics YOLOv8 [16] and YOLO11 [17] are anchor-free and decouple the classification and box-regression heads; box regression uses a distribution focal loss (DFL) over discretised offsets. Their segmentation variants follow YOLACT [18]: the network predicts a small set of full-image mask prototypes and, for every detected instance, a vector of coefficients; the instance mask is the sigmoid of the coefficient-weighted sum of prototypes, cropped to the box. This keeps mask prediction nearly as cheap as detection, unlike two-stage Mask R-CNN [19], which pools a feature map for each proposal.

YOLO11 introduces three blocks that matter for this project. The C3k2 block is a cross-stage partial bottleneck that carries edge and texture features through the backbone. The SPPF block pools the deepest backbone map at several receptive fields in sequence, giving multi-scale context at little cost. The C2PSA block adds position-sensitive self-attention to the deepest features, so the network can weigh where in the image the evidence lies. Section 4.1.4 distils features at exactly these three blocks.

Implication. A segmentation detector returns class, box, and mask in one pass. The mask gives the quality gate and the audit record the cap's exact region, which a whole-image classifier cannot provide.

2.3 Knowledge Distillation

Knowledge distillation trains a compact *student* to imitate a larger *teacher*. Hinton *et al.* [20] matched temperature-softened class probabilities (*response* or *logit* distillation), so that the student learns the teacher's similarity structure between classes. FitNets [21] introduced *hint* learning: a learned regressor maps an intermediate student layer onto a teacher layer and the

two are matched with an L_2 loss. Zagoruyko and Komodakis [22] transferred spatial attention maps instead of raw activations.

Distilling detectors is harder than distilling classifiers, because most image locations are background and dominate any loss summed over the feature map. Chen *et al.* [23] combined hint, logit, and bounded regression terms for two-stage detectors. Wang *et al.* [24] imitated features only near object anchors, and showed that full-map imitation can hurt because background swamps the signal. Normalising features before matching, or restricting the loss to informative regions, are the two common remedies.

Implication. For bottle caps, the decisive evidence is fine-grained (droplets, dents, a thread gap), so matching only the teacher's outputs would tell the student *what* to predict but not *where* to look. Phase 2 uses feature-level distillation at the three YOLO11 blocks above, with channel-wise L_2 normalisation to keep the background from dominating (Section 4.1.4).

2.4 Reduced-Precision and Accelerated Inference

Mixed-precision arithmetic stores weights and activations in 16-bit floating point while keeping numerically sensitive operations in 32 bits [25]. Integer quantisation goes further, replacing floating-point arithmetic with 8-bit integers [26]; both can change accuracy as well as speed, so each must be measured on the target hardware. NVIDIA TensorRT [27] compiles a network graph for a specific GPU: it fuses layers, selects kernels by timing them, and supports FP16 and INT8 precision with static or dynamic input shapes. The ONNX format and ONNX Runtime [28] provide a vendor-neutral intermediate representation and a portable CPU/GPU runtime.

Implication. A TensorRT engine is tied to the GPU and library version that built it. The deployment therefore compiles engines on the target machine, caches them under a version tag, and falls back to ONNX Runtime and then PyTorch when no compatible GPU is present (Section 4.2.5).

2.5 Class Imbalance and Evaluation

Defect data are usually imbalanced because compliant products far outnumber defective ones. Standard remedies are re-weighting or re-sampling the training data [29], class-balanced losses based on the effective number of samples [30], and focal loss, which down-weights easy examples [31]. Mixup [32] and CutMix [33] regularise training but can blend away a defect that covers a small part of the object. The benefit of any remedy is best judged with class-sensitive metrics: per-class precision and recall, Cohen's κ [34], and the Matthews correlation coefficient [35]. When an error count is zero, an exact binomial (Clopper–Pearson) bound [36] states how large the true rate could still be.

Implication. Phase 2 reports per-class AP, κ, MCC, and exact confidence bounds on false-accept and false-reject rates instead of a single accuracy figure.

2.6 Anomaly Detection for Printed Labels

Supervised detectors need labelled examples of every defect type. For printed labels, defects are rare and open-ended, so unsupervised anomaly detection, which learns only from good samples, is the usual alternative. The MVTec AD benchmark [37] established the task of image- and pixel-level anomaly detection on industrial objects and textures. PatchCore [38] stores a coreset of pretrained patch features from good images and scores a test patch by its distance to the nearest stored feature; it is among the most accurate methods, but its memory bank grows with the training set and nearest-neighbour search slows inference. EfficientAD [39] trains a lightweight student network to reproduce a pretrained teacher's features on good images, and an autoencoder to model global layout; disagreement between student and teacher, and between student and autoencoder, produces a pixel-level anomaly map at millisecond-level latency. The Anomalib library [40] provides reference implementations of both.

Localising the regions to inspect is a separate problem. Template matching by normalised cross-correlation [41, 42] finds a known pattern in an image without training data and is reliable when the pattern's appearance and scale are stable. Learned detectors are more flexible, but very large images must either be down-scaled, which destroys small structures, or sliced into tiles as in SAHI [43]. Generative models such as StyleGAN2-ADA [44] can synthesise additional good samples when training data are limited.

Implication. For a fixed label layout scanned at high resolution, template matching gives deterministic localisation without labelled boxes, and EfficientAD gives per-region anomaly maps from good samples only. Phase 2 tested both PatchCore and a YOLO-plus-SAHI localiser before settling on this combination (Section 3.2.3).

2.7 Limitations of Existing Systems

Table 2.2 summarises the gaps that the Phase 2 objectives address.

Table 2.2: Limitations of existing systems and the Phase 2 response

Limitation	**Evidence and Phase 2 response**	**Obj.**
Validation-only accuracy	Most packaging studies report validation accuracy [3, 7]. Phase 2 reports held-out test results for FP32 and deployed FP16 models and product-level FAR/FRR with exact bounds.	Primary, O1
No threshold analysis	Decision thresholds are rarely justified. Phase 2 sweeps the gate thresholds and reports the margin to the nearest error.	Primary
Whole-image fill classification	Fill level is classified, not measured [6]. Phase 2 localises the meniscus, computes a ratio, and derives its sensitivity to pixel error.	O2
Heavy models at the edge	Accurate backbones are slow [5]. Phase 2 distils a 62 M-parameter teacher into a 22 M student and compiles FP16 engines.	Primary, O3
Scarce defect labels	Rare, varied defects [4, 13]. The label prototype trains on good samples only.	O4
No auditable deployment	Few systems store evidence per product. Phase 2 stores raw frames, labels, and verdicts in SQLite and exposes them in a dashboard.	O5

These gaps lead to the architecture in Chapter 3: distilled segmentation for caps, geometric measurement for fill level, good-only anomaly detection for labels, and an optimised, auditable deployment around all three.

Chapter 3

Methodology

This chapter describes the proposed system at the level of modules and data flow, the plan that was followed and how it changed, and the resources used. Algorithms, equations, and pseudocode are deferred to Chapter 4.

3.1 System Architecture

3.1.1 Design principles

Three principles guided the Phase 2 architecture.

1. **One station, one question.** Each station answers one inspection question (cap state, fill level, or label integrity) with its own models, decision rule, audit database, and API namespace. Stations share a server and a dashboard only for convenience, so one can be changed or removed without touching the others.

2. **Asymmetric, fail-safe decisions.** A doubtful product is never passed automatically. Every station has four outcomes: `PASS`, `REJECT`, `UNCERTAIN` (sent to human review without ejection), and `INSPECTION_FAILED` (the input could not be evaluated).

3. **Evidence before verdict.** Every verdict is stored with the raw frames, the model outputs, and the reason for the decision, so any automated decision can be audited and the stored images can be re-used for training.

3.1.2 Architecture overview

Figure 3.1 shows the complete system. The input layer acquires frames: three synchronised views per bottle for the cap station, one side view for the fill station, and a full unwrapped scan for the label prototype. The inference layer runs each station's models as TensorRT engines (cap and fill) or ONNX models (label). The decision layer turns model outputs into verdicts with station-specific rules. The persistence and presentation layer writes each result to SQLite and to disk, pushes it to the dashboard over a WebSocket, and drives the reject actuator.

3.1.3 Station 1: cap inspection

The three camera views of one bottle are resized to the network input and passed through the cap network as one batch of three. Non-maximum suppression (NMS) is applied to each view separately. The network is a YOLO11m instance-segmentation *student* that was trained with feature-level distillation from a frozen YOLO11x-seg *teacher* (Section 4.1.4); the teacher and the distillation projectors exist only during training. Each view is judged on its own, and the bottle verdict is *defect-dominant*: a defect seen by any camera rejects the bottle. This design replaces the Phase 1 cascade of a bottle/cap detector followed by a MobileNetV3-Small crop classifier; the reasons are given in Section 5.5.1.

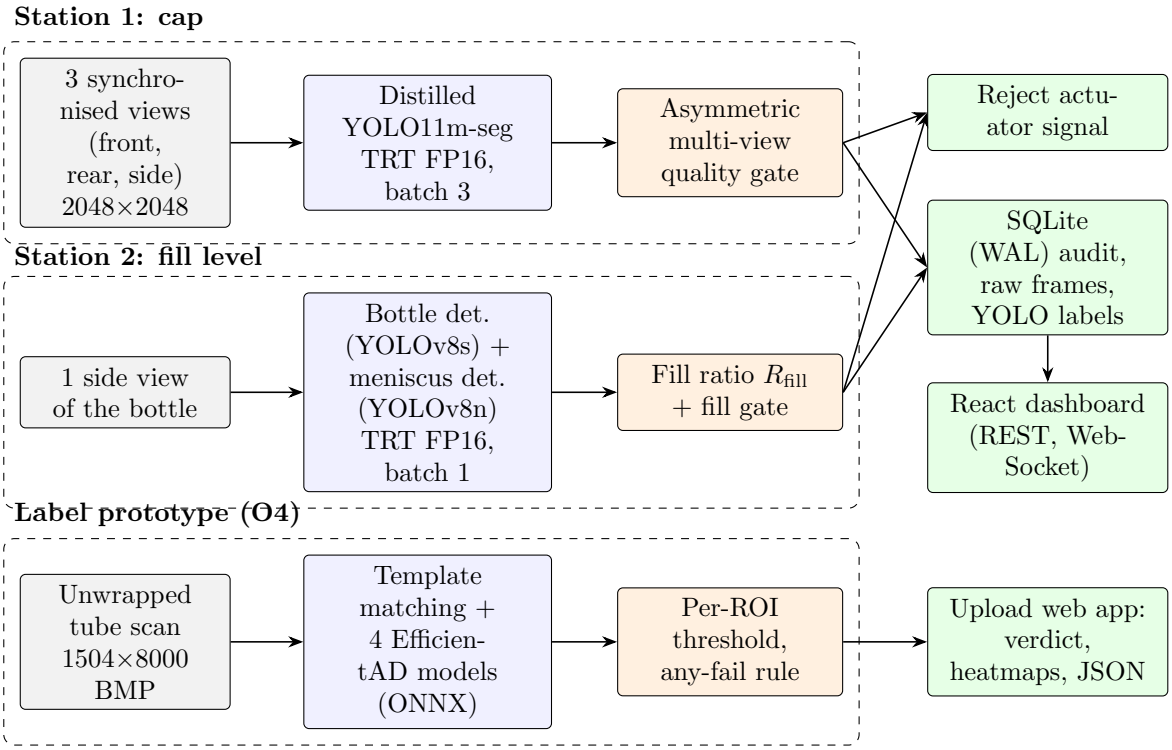


Figure 3.1: Overall VisionFood QAI architecture. Each row is an independent station with its own models and decision rule; the cap and fill stations share the audit and dashboard layer, and the label prototype runs as a separate upload-and-inspect application. Blue: inference; orange: decision; green: outputs.

3.1.4 Station 2: fill level

Figure 3.2 shows the fill station as a multi-stage pipeline. Stage 1 localises the bottle with a YOLOv8s detector and keeps the highest-scoring bottle box. Stage 2 detects the liquid surface (meniscus) with a separate YOLOv8n detector run on the full frame, associates a surface box with the bottle, trims the bottle box to remove detector padding, and computes the fill ratio R_{fill} in closed form. Stage 3 maps the ratio into one of three fill classes (underfill, acceptable, overfill) and then into a gate verdict, with a review band around each limit when the meniscus detection is weak. The limits are configuration values, not learned parameters.

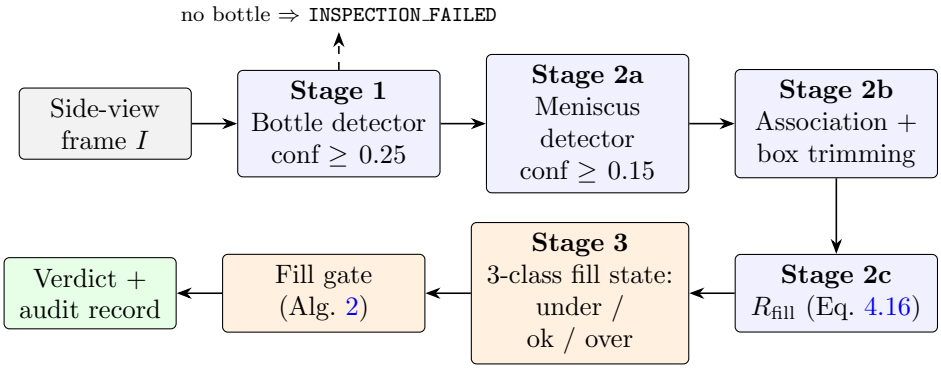


Figure 3.2: Multi-stage fill-level pipeline (Station 2). Stage 1 localises the bottle, Stage 2 localises the meniscus and measures the ratio, and Stage 3 classifies the fill state and applies the review band.

3.1.5 Label prototype (O4)

The label prototype inspects the printed label of the Himalaya Winter Defense Moisturizing Cream (50 ml) tube. The input is a flat, unwrapped scan of about 1504×8000 pixels. Because the label wraps around a cylinder, each printed region appears twice in the scan. Figure 3.3 shows the three steps: (i) *preprocessing and localisation*, in which the scan is converted to grayscale and four full-region templates are located by coarse-to-fine normalised cross-correlation, keeping the complete (uncut) occurrence of each region; (ii) *ROI cropping*, in which each located region is cut out and rotated upright; and (iii) *model routing*, in which each crop is sent to the anomaly model trained for that region. A label fails if any region scores above its calibrated threshold or if any region cannot be located.

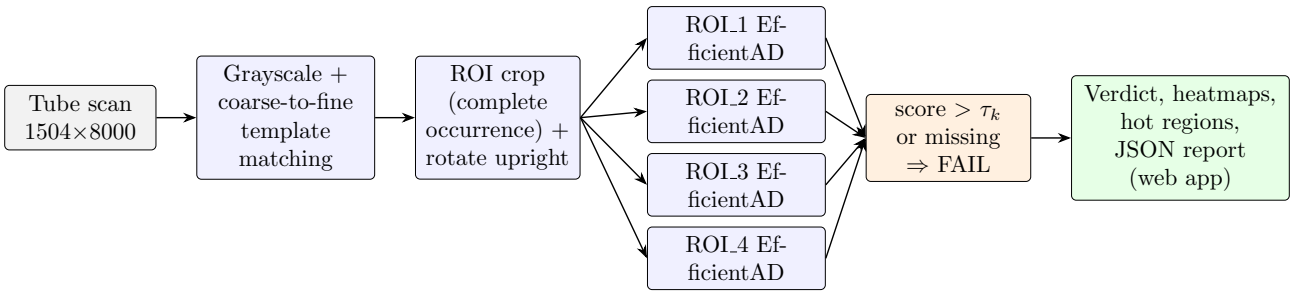


Figure 3.3: Label-defect prototype: template-matching localisation, ROI cropping, and routing of each crop to its own anomaly model.

The four regions are: ROI_1, the logo and product name; ROI_2, the ingredient line and paragraph; ROI_3, the address, regulatory numbers, manufacturing and expiry dates, and barcode; and ROI_4, the "3 Way Care" graphic.

3.1.6 Deployment architecture

Figure 3.4 shows how the cap and fill stations are served. A FastAPI application exposes Station 1 under `/api/*` and Station 2 under `/api/fill/*`, with one WebSocket stream per station. Each station is owned by a service object that holds the engine, the gate, the audit logger, and the overlay renderer. Only one station's engines occupy GPU memory at a time: the cap engine is loaded at start-up, the fill engines are loaded on demand, and a station switch in the dashboard unloads one set before loading the other. The backend and an nginx container serving the built dashboard are started together with Docker Compose.

3.1.7 Data sources and repositories

Table 3.1 lists every dataset used. The code and data are kept in the following locations:

- Repository: https://github.com/ShreyasBairyKS/Capstone-Project (cap and fill stations, dashboard, deployment).
- Label prototype branch: https://github.com/ShreyasBairyKS/Capstone-Project/tree/himalaya-label-detection.

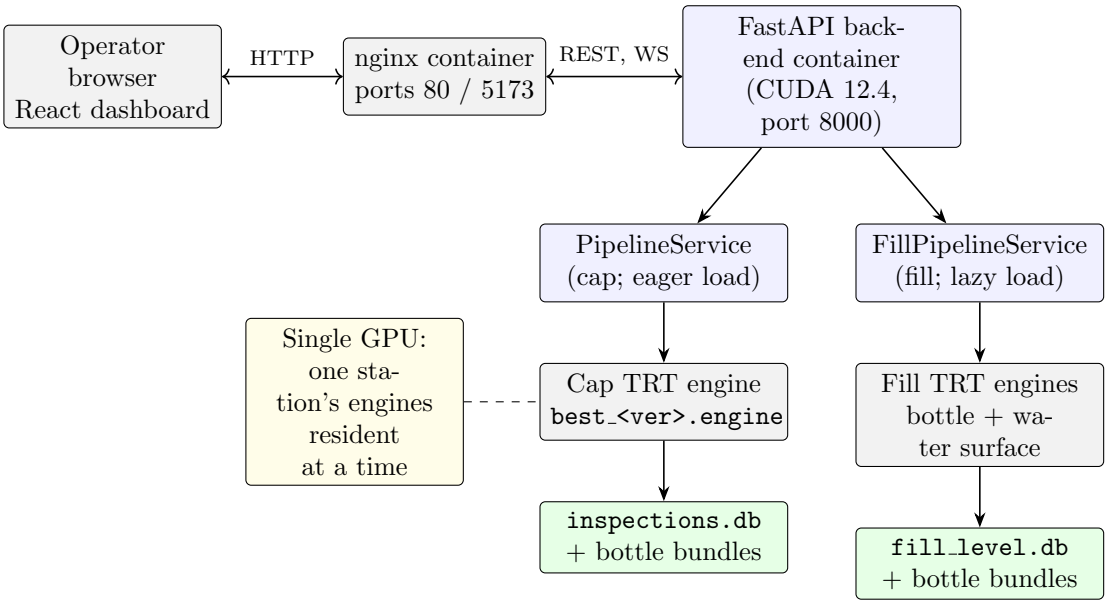


Figure 3.4: Deployment architecture. Two Docker containers (dashboard and backend) share bind-mounted model and audit folders with the host. Station switching unloads one engine set before loading the other.

- Local project folder `E:\P-25 Vision Food ai`, which holds the NSC scans in `dataset\NSC` and the label-prototype working tree in `himalaya_label_detection`.

The NSC scans are not redistributed because the product images belong to the manufacturer.

Table 3.1: Datasets used in Phase 2

Dataset	**Used for**	**Content and split**	**Licence / source**
`bottle_cap_sdp` v7 [45]	Station 1 teacher and student	6 cap classes, polygon masks; 7,458 / 712 / 354 images (train / valid / test), 2048×2048	Roboflow Universe, CC BY 4.0
Merged bottle/cap sets	Station 2 bottle detector	4 public sources; 2,286 / 417 / 280 images; 2,133 / 774 / 356 bottle boxes	Public, CC BY 4.0
`fill-level` v1 [46] + `bottle-liquid-level` v2 [47]	Station 2 meniscus detector	Single class `water_surface`; 545 / 31 / 41 images; 1,683 / 54 / 76 boxes	Roboflow Universe, CC BY 4.0
Four-class cap classification set	Model-selection benchmark (Table 5.12)	7,540 images; 308-image test split	Earlier project data
Himalaya NSC	Label prototype (O4)	120 unwrapped tube scans, ≈ 1504×8000 BMP: 78 good, 42 defective	Industrial partner (local copy)

3.2 Project Planning

3.2.1 Gantt chart

Figure 3.5 shows the Phase 2 timeline from June to November 2026, reconstructed from the repository's commit history so that it records what actually happened rather than a plan drawn after the fact. Work ran in three parallel tracks: the label prototype (June to October), the cap

station (September to October), and integration, deployment, and documentation (September to November). The dependencies are: the student cannot be trained before the teacher; TensorRT engines and the stress test require the integrated pipeline; and Station 2 integration requires the station-switching mechanism.

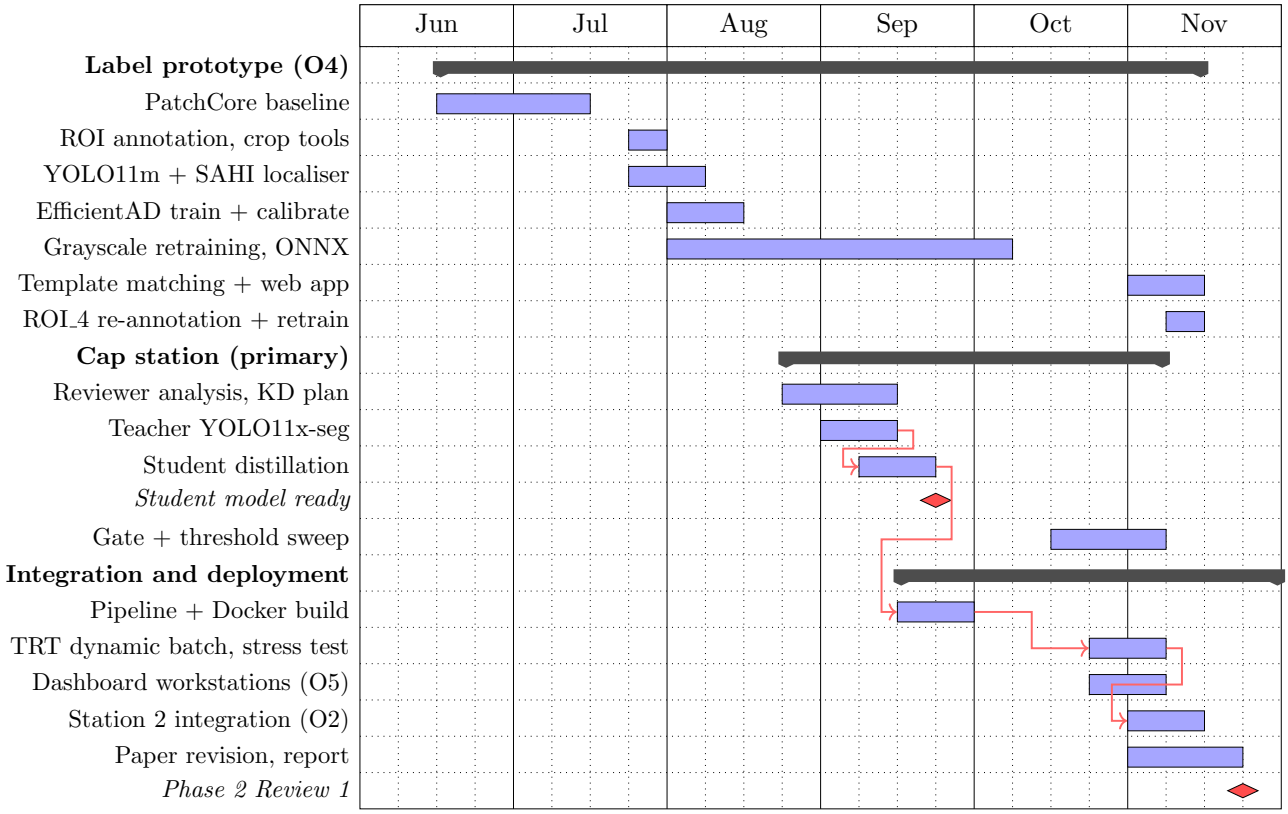


Figure 3.5: Phase 2 Gantt chart (weekly resolution, June–November 2026), reconstructed from the repository commit history. Red arrows mark dependencies; diamonds mark milestones.

3.2.2 Planned versus actual progress

Table 3.2 compares the Phase 1 plan with what Phase 2 delivered. Several Phase 1 components were deliberately dropped or replaced after the Phase 1 evaluation and the conference reviews; the reasons are recorded alongside.

3.2.3 Risk management

Table 3.3 lists the risks identified during Phase 2, the mitigation applied, and the evidence that the plan was adjusted.

3.3 Software and Hardware Requirements

Tables 3.4 and 3.5 list the resources used. Ultralytics was chosen because one library covers training, segmentation, and export for YOLOv8 and YOLO11; TensorRT because the target is an NVIDIA GPU; FastAPI because it serves REST and WebSocket endpoints from one asynchronous process; and SQLite because a single-file database with write-ahead logging needs no separate server on the line.

Table 3.2: Phase 1 plan versus Phase 2 outcome

Phase 1 plan item	Status	Phase 2 outcome and reason
YOLOv8 detector + EfficientViT-M5 / MobileNetV3 cap classifier	Replaced	Distilled YOLO11m-seg student; classifier reached 86.5% and gave no location (Section 5.5.1).
Fill-level ratio from water-surface detector	Completed	Formalised with trimming, association, review band, sensitivity analysis, and measured error (O2).
Sub-80 ms latency per frame	Partly met	7.4 ms network time for three views; closed loop 299 ms because of synchronous storage (O3).
ONNX edge deployment	Extended	TensorRT FP16 engines with ONNX Runtime and PyTorch fallbacks; Docker deployment.
Label misalignment and packaging damage	Re-scoped	Label-defect prototype on the Himalaya NSC tube (O4).
Monte Carlo Dropout uncertainty	Replaced	Deterministic `UNCERTAIN` band in each gate; no sampling cost at inference.
REMEDY routing, SKU profiles, four package types	Dropped	Not evaluable within Phase 2; reviewers asked for scope to match tested components.
React dashboard, inspection history, PDF shift report	Partly met	Dashboard, audit table, replay, review queue, and CSV export delivered; PDF shift report not implemented (O5).

Table 3.3: Risk register with mitigation and evidence of adjustment

Risk	Impact	Mitigation and evidence of adjustment
Cap classifier below target (86.5%)	High	Switched to segmentation; distilled a large teacher into a deployable student (Sections 5.5.1, 4.1.4).
Large model too slow for line rate	High	Feature distillation (2.8× fewer parameters) and TensorRT FP16 engines; latency measured per stage (Table 5.10).
TensorRT static batch silently fell back to single-image inference	Medium	Rebuilt engines with a dynamic batch profile, added batched warm-up checks and versioned engine files (September–October 2026).
Two stations exceeding one GPU's memory	Medium	Lazy loading of Station 2 and explicit unload/load on station switch, guarded by locks.
PatchCore too slow for the label scans	Medium	Replaced by EfficientAD-Medium after the June–July baseline.
Down-scaled 8000 px scans hid label regions from the detector	Medium	Tried YOLO11m with SAHI tiling (July); replaced by deterministic template matching, which located all four regions on all 120 scans (October).
White label background caused false alarms in colour models	Medium	Retrained on grayscale crops; recall on ROI_1–3 rose to 93–94% (Table 5.9).
Noisy ROI_4 annotations	Medium	Re-annotated and retrained ROI_4 on combined data; still open (57.9% precision).
Synchronous audit storage limits throughput	Medium	Measured and reported (Table 5.11); asynchronous writes planned as future work.
Conference rejection (CSITSS 2026)	Medium	Reviewer comments mapped to concrete changes; paper rewritten around measured stations.

Table 3.4: Software requirements

Category	Tool / library	Version
Language	Python	3.12 (host); 3.11 (container)
Deep learning	PyTorch (CUDA 12.1 build)	2.4.1
Detection / segmentation	Ultralytics	8.4.192 (deployment)
Inference engine	NVIDIA TensorRT	10.16.1
Portable runtime	ONNX, ONNX Runtime	1.16+, 1.31.1
Anomaly detection	Anomalib (EfficientAD-M)	1.1.0
Computer vision	OpenCV	4.9–4.10
Backend	FastAPI, Uvicorn, Pydantic	0.111+, 0.30+, 2.8+
Frontend	React, Vite, Recharts	19.2, 8.3, 3.10
Database	SQLite (WAL journal)	bundled with Python
Containers	Docker Compose, nginx, NVIDIA Container Toolkit	—
Base image	`nvidia/cuda:12.4.1-cudnn-runtime-ubuntu22.04`	12.4.1
Annotation, version control	Roboflow, LabelImg, Git / GitHub	—

Table 3.5: Hardware requirements

Component	Specification (measurement workstation)
GPU	NVIDIA RTX A5000, 24 GB VRAM
CPU	Intel Xeon W-2295, 18 cores / 36 threads
System RAM	128 GB (training workstation)
Cameras (Station 1)	Three synchronised views per bottle; 2048×2048 capture
Cameras (Station 2)	One side view per bottle
Label scanner (O4)	Line-scan or flatbed capture of the unwrapped tube, ≈ 1504×8000
OS	Windows 11 host with Docker (Linux containers)

Chapter 4

Implementation

This chapter gives the algorithms behind each station, the loss functions used in training, the decision rules used at inference, the way the modules are integrated and deployed, and the tests that verify them. Long code listings are placed in Appendix A.

4.1 Algorithm Implementation

4.1.1 Notation

Table 4.1 collects the symbols used throughout this chapter.

Table 4.1: Notation

Symbol	Meaning
$I_v \in \mathbb{R}^{H\times W\times 3}$	Image from camera view $v \in \{1,2,3\}$
$\mathcal{D}_v$	Detections $\{(b_i, c_i, s_i, M_i)\}$ in view v: box $b_i = (x_1, y_1, x_2, y_2)$, class c_i, score s_i, mask M_i
$\mathcal{C}$, $\mathcal{C}_d$	Cap classes $\mathcal{C} = \{\text{good}, \text{damaged}, \text{misplaced}, \text{no}, \text{open}, \text{wet}\}$; defect classes $\mathcal{C}_d = \mathcal{C} \setminus \{\text{good}\}$
F_ℓ^T, F_ℓ^S	Teacher and student feature maps at distillation site ℓ
ϕ_ℓ	Learned 1×1 convolutional projector at site ℓ (training only)
$\alpha(t)$	Distillation weight at epoch t
τ_p, τ_d, τ_u	Cap gate thresholds: pass, defect, uncertain floor
b, w	Bottle box $b = (x_1^b, y_1^b, x_2^b, y_2^b)$ and associated meniscus (water-surface) box w
h, h'	Bottle box height before and after trimming
y_w	Meniscus height (box mid-line, clamped)
R_{fill}	Fill ratio, 0 = empty, 1 = full
$\theta_u, \theta_o, \beta, c_{\min}$	Underfill and overfill limits, review half-band, minimum meniscus confidence
T_k, $\rho_k(x, y)$	Template of label region k and its normalised cross-correlation map
A_k, σ_k, τ_k	Anomaly map, anomaly score, and threshold of label region k

4.1.2 Segmentation network

Both cap networks (teacher YOLO11x-seg and student YOLO11m-seg) share the YOLO11 segmentation design [17]. For an input image the backbone and neck produce feature maps at strides 8, 16, and 32. At every anchor point the head predicts (i) class logits $z \in \mathbb{R}^{|\mathcal{C}|}$, (ii) a discretised distribution over the four box-edge offsets, and (iii) a mask-coefficient vector $\mathbf{m} \in \mathbb{R}^K$. A separate prototype branch predicts $K = 32$ full-image prototypes $P \in \mathbb{R}^{K\times H_p\times W_p}$. Following YOLACT [18], the mask of instance i is

$$M_i = \sigma\left(\sum_{k=1}^{K} m_{i,k}\, P_k\right) \odot \mathbb{1}[b_i], \tag{4.1}$$

where σ is the logistic function and $\mathbb{1}[b_i]$ crops the mask to the predicted box. Because the prototypes are shared, the per-instance cost of a mask is one K-dimensional linear combination.

4.1.3 Supervised training loss

The base loss is the Ultralytics segmentation loss, a weighted sum of four terms:

$$\mathcal{L}_{\text{YOLO}} = \lambda_{\text{box}}\,\mathcal{L}_{\text{CIoU}} + \lambda_{\text{box}}\,\mathcal{L}_{\text{seg}} + \lambda_{\text{cls}}\,\mathcal{L}_{\text{BCE}} + \lambda_{\text{dfl}}\,\mathcal{L}_{\text{DFL}}, \tag{4.2}$$

where $\mathcal{L}_{\text{CIoU}}$ is the complete-IoU box loss, $\mathcal{L}_{\text{seg}}$ is the per-pixel binary cross-entropy between predicted and ground-truth masks (normalised by box area and weighted by the box gain, as implemented in Ultralytics), $\mathcal{L}_{\text{BCE}}$ is the binary cross-entropy classification loss, and $\mathcal{L}_{\text{DFL}}$ is the distribution focal loss on the edge distributions. Positive anchors are chosen by a task-aligned assigner. We used $\lambda_{\text{box}} = 7.5$, $\lambda_{\text{dfl}} = 1.5$, and $\lambda_{\text{cls}} = 4.0$.

The classification gain was raised from the default 0.5 to compensate for class imbalance. With n_c training instances of class c, the majority-to-class ratio averaged over classes is

$$\bar{r} = \frac{1}{|\mathcal{C}|}\sum_{c\in\mathcal{C}} \frac{\max_{c'} n_{c'}}{n_c} = 6.29, \tag{4.3}$$

and the gain was set to $\min(\bar{r}, 4.0) = 4.0$. This is a single scalar: it strengthens the classification signal relative to localisation but does not re-weight classes individually, unlike class-balanced or focal losses [30, 31].

4.1.4 Teacher–student feature distillation

General form. A distillation objective for a detector can combine a response term, a box term, and a feature term:

$$\mathcal{L}_{\text{distill}} = \alpha\,\mathcal{L}^{\text{KD}}_{\text{cls}} + \beta\,\mathcal{L}^{\text{KD}}_{\text{box}} + \gamma\,\mathcal{L}^{\text{KD}}_{\text{feat}}. \tag{4.4}$$

The response (logit) term matches temperature-softened class distributions [20],

$$\mathcal{L}^{\text{KD}}_{\text{cls}} = \mathcal{T}^2\,\text{KL}\Big(\text{softmax}(z^T/\mathcal{T})\,\Big\|\,\text{softmax}(z^S/\mathcal{T})\Big), \tag{4.5}$$

with temperature $\mathcal{T} > 1$; the box term matches the teacher's edge distributions in the same way; and the feature term matches intermediate representations through learned projectors [21].

Configuration used. For bottle caps the decisive evidence (droplets, dents, a thread gap) covers a small part of the cap, and the teacher and student make near-identical final predictions on most images. Matching outputs alone would mainly transfer *what* to predict. We therefore set $\alpha = \beta = 0$ and distil features only, adding them to the supervised loss of Eq. (4.2):

$$\mathcal{L} = \mathcal{L}_{\text{YOLO}} + \alpha(t)\sum_{\ell\in\mathcal{S}} \frac{1}{C_\ell H_\ell W_\ell}\left\|\widehat{F^T_\ell} - \widehat{\phi_\ell(F^S_\ell)}\right\|_2^2, \tag{4.6}$$

where $\mathcal{S} = \{\text{C3k2},\ \text{SPPF},\ \text{C2PSA}\}$ are the three distillation sites, $C_\ell \times H_\ell \times W_\ell$ is the teacher feature shape at site ℓ, and $\widehat{\cdot}$ denotes channel-wise L_2 normalisation at every spatial position

(x, y):

$$\widehat{F}(x, y) = \frac{F(x, y)}{\|F(x, y)\|_2 + \epsilon}. \tag{4.7}$$

Why normalise. For unit vectors $\hat{a}$ and $\hat{b}$,

$$\|\hat{a} - \hat{b}\|_2^2 = 2\,(1 - \cos(a, b))\,, \tag{4.8}$$

so the feature term of Eq. (4.6) is a mean cosine distance between teacher and projected student features. It depends on the *direction* of each feature vector and not on its magnitude, so the large, uniform background, where activations are strong but uninformative, cannot dominate the loss. This addresses the background-swamping problem noted for detector distillation [24].

Distillation sites. The three sites were chosen for what they carry. C3k2 at the backbone output holds edge and texture features, including thread and ridge geometry. SPPF at the backbone–neck transition holds multi-scale pooled context, which helps fine cues survive down-sampling. C2PSA holds the teacher's spatial attention, which transfers *where* the fine defect evidence lies. The student and teacher feature maps have the same spatial size at each site because the two models share strides; only the channel widths differ. The projectors ϕ_ℓ are 1×1 convolutions that lift student channels to teacher width ($256 \to 512$ and $512 \to 1024$).

Schedule. The distillation weight ramps linearly over the first 20 epochs:

$$\alpha(t) = \alpha_0 + (\alpha_1 - \alpha_0)\,\min\!\left(\frac{t}{t_r}, 1\right), \qquad \alpha_0 = 0.05,\ \alpha_1 = 0.5,\ t_r = 20. \tag{4.9}$$

Early epochs are dominated by the supervised loss, so the student first learns basic detection; the alignment pressure then reaches full strength. Teacher features are detached, so no gradient reaches the teacher. The projectors exist only in training and are discarded before export, so distillation adds no inference cost. Algorithm 1 summarises one training step.

Algorithm 1 One training step of feature distillation

Input: Batch (X, Y), frozen teacher f^T, student f^S, projectors $\{\phi_\ell\}$, epoch t
Output: Updated student and projector parameters

Register forward hooks on sites $\ell \in \mathcal{S}$ in f^T and f^S
$\{F_\ell^T\} \leftarrow f^T(X)$ **without gradient** — *teacher frozen*
$(\hat{Y}, \{F_\ell^S\}) \leftarrow f^S(X)$
$\mathcal{L}_{\text{YOLO}} \leftarrow$ Eq. (4.2) on $(\hat{Y}, Y)$
$\mathcal{L}_{\text{feat}} \leftarrow 0$
for each site $\ell \in \mathcal{S}$ **do**
    $G \leftarrow \phi_\ell(F_\ell^S)$ — *1×1 projection to teacher width*
    $\mathcal{L}_{\text{feat}} \leftarrow \mathcal{L}_{\text{feat}} + \text{MSE}\big(\text{norm}(F_\ell^T),\ \text{norm}(G)\big)$ — *Eq. (4.7)*
end for
$\mathcal{L} \leftarrow \mathcal{L}_{\text{YOLO}} + \alpha(t)\,\mathcal{L}_{\text{feat}}$ — *Eq. (4.9)*
Back-propagate $\mathcal{L}$; update f^S and $\{\phi_\ell\}$ with AdamW

Training configuration. Table 4.2 lists the settings. The teacher was trained first on the same data and settings at batch 16 for the full 150 epochs and then frozen. The student was initialised from COCO weights [48].

Table 4.2: Training configuration of the cap teacher and student

Setting	**Teacher (YOLO11x-seg)**	**Student (YOLO11m-seg)**
Parameters / GFLOPs @640	62.06 M / 297.4	22.34 M / 113.0
Input size, batch	640 px, 16	640 px, 32
Optimiser	AdamW, lr $10^{-3} \rightarrow 10^{-5}$ cosine, 5 warm-up epochs, weight decay $5{\times}10^{-4}$	
Loss gains	box 7.5, DFL 1.5, cls 4.0	
Augmentation	mosaic (off for last 10 epochs), copy-paste 0.5, mixup 0.15, rotation $\pm 15°$, translate 0.1, scale 0.5, flips (v 0.3, h 0.5), HSV jitter, RandAugment, erasing 0.4	
Distillation	—	Eq. (4.6), sites C3k2/SPPF/C2PSA
Epochs (best)	150 (139)	126, early stop, patience 30 (96)
Wall-clock time	12.2 h	5.3 h
Hardware	NVIDIA RTX A5000 24 GB; mixed precision; seed 42	

4.1.5 Asymmetric multi-view quality gate

The cap engine runs with a confidence floor of 0.35. Let $\mathcal{D}_v$ be the detections of view v above the floor, $(c_v^\star, s_v^\star)$ the class and score of the top detection, and

$$s_v^d = \max\{s_i : (b_i, c_i, s_i, M_i) \in \mathcal{D}_v,\ c_i \in \mathcal{C}_d\} \tag{4.10}$$

the strongest defect score ($-\infty$ if none). The per-view verdict is

$$g(v) = \begin{cases} \texttt{FAILED} & \mathcal{D}_v = \emptyset \\ \texttt{REJECT} & s_v^d \geq \tau_d \\ \texttt{PASS} & c_v^\star = \text{good},\ s_v^\star \geq \tau_p \\ \texttt{UNCERTAIN} & \text{otherwise} \end{cases} \tag{4.11}$$

with defaults $\tau_d = 0.45$, $\tau_p = 0.75$, and an uncertain floor $\tau_u = 0.40$ that labels a good-cap score in $[\tau_u, \tau_p)$ as borderline in the audit record. The bottle verdict aggregates the views with a strict precedence:

$$G = \begin{cases} \texttt{REJECT} & \exists v : g(v) = \texttt{REJECT} \\ \texttt{INSPECTION_FAILED} & \text{else } \exists v : g(v) = \texttt{FAILED} \\ \texttt{UNCERTAIN} & \text{else } \exists v : g(v) = \texttt{UNCERTAIN} \\ \texttt{PASS} & \text{otherwise.} \end{cases} \tag{4.12}$$

`REJECT` and `INSPECTION_FAILED` fire the actuator; `UNCERTAIN` goes to the review queue without ejection. The thresholds are asymmetric by design ($\tau_d < \tau_p$): the bar for rejecting is lower than the bar for passing, so a doubtful cap is never passed automatically. All three thresholds can be changed at run time from the dashboard.

4.1.6 Fill-level estimation

Stage 1: bottle localisation. The bottle detector runs at confidence ≥ 0.25 and the highest-scoring `bottle` box b is kept. If none is found the frame is treated as a conveyor gap.

Stage 2: meniscus association and ratio. The meniscus detector runs on the full frame at confidence ≥ 0.15. A candidate surface box $w = (x_1^w, y_1^w, x_2^w, y_2^w)$ is associated with the bottle when

$$x_1^b \leq \tfrac{1}{2}(x_1^w + x_2^w) \leq x_2^b \quad \wedge \quad \left(y_1^w \in [y_1^b, y_2^b] \;\vee\; y_2^w \in [y_1^b, y_2^b]\right), \tag{4.13}$$

and among associated candidates the highest-scoring one is kept. With $h = y_2^b - y_1^b$, the bottle box is trimmed to remove detector padding at the cap and the base:

$$y_1' = y_1^b + \lfloor 0.03h \rfloor, \qquad y_2' = y_2^b - \lfloor 0.02h \rfloor, \qquad h' = y_2' - y_1'. \tag{4.14}$$

The meniscus height is the mid-line of the surface box, clamped to the trimmed bottle,

$$y_w = \min\Big(y_2',\ \max\big(y_1',\ \lfloor (y_1^w + y_2^w)/2 \rfloor\big)\Big), \tag{4.15}$$

and the fill ratio is (image y grows downwards)

$$R_{\text{fill}} = \frac{y_2' - y_w}{h'} \in [0, 1]. \tag{4.16}$$

Figure 4.1 illustrates the geometry.

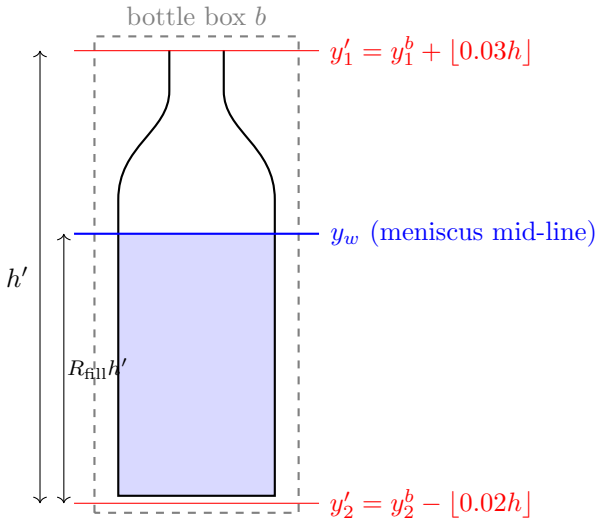


Figure 4.1: Fill-ratio geometry: trimmed bottle box (red) and meniscus mid-line (blue).

Stage 3: three-class fill state and gate. The ratio is mapped to a fill state with the configurable limits $\theta_u = 0.40$ and $\theta_o = 0.85$:

$$\text{state}(R_{\text{fill}}) = \begin{cases} \text{underfill} & R_{\text{fill}} < \theta_u \\ \text{overfill} & R_{\text{fill}} > \theta_o \\ \text{acceptable} & \text{otherwise.} \end{cases} \tag{4.17}$$

These limits are engineering defaults, not regulatory values; they are read from a single configuration source so that a plant can set them per product, and the gate, the overlay ruler, and the health endpoint all read the same values. Algorithm 2 applies a review band of half-width $\beta = 0.02$ around each limit when the meniscus confidence c_w is below $c_{\min} = 0.50$.

Algorithm 2 Fill gate

Input: Bottle box b (or none), meniscus box w with confidence c_w (or none), R_{fill}
Output: Verdict V and actuator flag

if no bottle **then**
  return `INSPECTION_FAILED`, no actuation — *conveyor gap*
else if no associated meniscus **then**
  return `REJECT` — *empty or opaque*
else if $\min_{\theta \in \{\theta_u, \theta_o\}} |R_{\text{fill}} - \theta| \le \beta$ **and** $c_w < c_{\min}$ **then**
  return `UNCERTAIN` — *borderline and weakly detected*
else if $\text{state}(R_{\text{fill}}) \neq$ acceptable **then**
  return `REJECT` (underfill or overfill)
else
  return `PASS`
end if

Sensitivity. Differentiating Eq. (4.16) gives

$$\frac{\partial R_{\text{fill}}}{\partial y_w} = -\frac{1}{h'}, \qquad \frac{\partial R_{\text{fill}}}{\partial y_1'} = \frac{R_{\text{fill}}}{h'}, \qquad \frac{\partial R_{\text{fill}}}{\partial y_2'} = \frac{1 - R_{\text{fill}}}{h'}. \tag{4.18}$$

An error of δ pixels in the meniscus shifts the ratio by δ/h'. For the median test bottle ($h = 620\,\text{px}$, $h' \approx 589\,\text{px}$) one pixel is 0.0017, so the ± 0.02 review band corresponds to about 12 pixels. If the three edge errors are independent with standard deviations σ_w, σ_1, and σ_2, first-order propagation gives

$$\sigma_R \approx \frac{1}{h'} \sqrt{\sigma_w^2 + R_{\text{fill}}^2 \sigma_1^2 + (1 - R_{\text{fill}})^2 \sigma_2^2}. \tag{4.19}$$

Using the mid-line of the surface box rather than one of its edges halves the effect of a symmetric error in the box height, because both edges move by equal and opposite amounts.

Why a rule rather than a classifier. A MobileNetV3 fill-level classifier was also trained during the project and reached 93.10% test accuracy (135 of 145 images). It is not used in

the deployed path: a classifier returns a label but no measurement, so its limits cannot be moved per product without retraining, and its errors cannot be traced to a pixel position. The geometric rule exposes R_{fill}, its limits, and its sensitivity.

4.1.7 Label-defect prototype

Preprocessing and localisation. The scan I is converted to grayscale. For each region k a full-resolution template T_k was cut from one clean reference scan. The match score at offset (x, y) is the zero-mean normalised cross-correlation [41]

$$\rho_k(x,y) = \frac{\sum_{u,v} \tilde{T}_k(u,v)\, \tilde{I}(x+u, y+v)}{\sqrt{\sum_{u,v} \tilde{T}_k(u,v)^2 \;\sum_{u,v} \tilde{I}(x+u, y+v)^2}} \in [-1, 1], \tag{4.20}$$

where $\tilde{\cdot}$ subtracts the mean over the template window. Search is coarse-to-fine: Eq. (4.20) is evaluated on the image and template down-sampled by 4, and the peak is refined at full resolution within ±12 pixels. Because the label wraps around the tube, each region occurs twice; candidates whose region would extend past the scan are discarded so that the complete occurrence is kept. A region is *missing* when its best score is below 0.45. If two located regions overlap by more than 10% of the smaller one, the lower-scoring region is searched again with the other's area excluded, so that a damaged region is reported missing instead of silently landing on a neighbour.

ROI cropping. Each located region is cropped and rotated 90° clockwise into the upright orientation in which the anomaly models were trained, then resized to 256×256. For the grayscale models the single channel is replicated to three channels.

Model routing and anomaly scoring. Each crop is routed to the EfficientAD-Medium model of its region [39]. EfficientAD has a frozen pretrained teacher network $\mathcal{T}$, a student $\mathcal{S}$ with two output groups, and an autoencoder $\mathcal{A}$, all trained only on good crops. For a crop x the two anomaly maps are

$$A^{\text{ST}}(x) = \frac{1}{C} \sum_c \big(\mathcal{T}_c(x) - \mathcal{S}^{\text{st}}_c(x)\big)^2, \qquad A^{\text{AE}}(x) = \frac{1}{C} \sum_c \big(\mathcal{A}_c(x) - \mathcal{S}^{\text{ae}}_c(x)\big)^2, \tag{4.21}$$

the first sensitive to local texture anomalies and the second to logical or layout anomalies. Each map is normalised with statistics from good validation crops and the two are averaged into $A_k(x)$. The region score is the mean of the map,

$$\sigma_k = \frac{1}{|\Omega|} \sum_{p \in \Omega} A_k(x)_p, \tag{4.22}$$

and the label verdict is

$$\text{label} = \begin{cases} \texttt{FAIL} & \exists k : \ \sigma_k > \tau_k \text{ or region } k \text{ missing} \\ \texttt{PASS} & \text{otherwise.} \end{cases} \tag{4.23}$$

Threshold calibration. Defective crops are used only to set thresholds. Two strategies were used: a recall-targeted threshold at the 5th percentile of defective-crop scores, which guarantees that at least 95% of the calibration defects exceed it, and an F1-optimal sweep. For the final ROI_4 model the crops were split 80/20 with a fixed seed; the threshold was set on the calibration part for a 95% recall target and evaluated once on the remaining part.

Explainability. The upload-and-inspect application overlays the pixels of A_k that exceed τ_k, finds up to three connected hot regions, and reports for each how much it adds to σ_k and what the region would score if that area were replaced by the crop median. If the counterfactual score falls below the threshold, that area alone causes the rejection. Algorithm 3 summarises the prototype.

Algorithm 3 Label-defect inspection

Input: Scan I, templates $\{T_k\}$, models $\{\mathcal{M}_k\}$, thresholds $\{\tau_k\}$
Output: Label verdict, per-region scores and heatmaps

$G \leftarrow \text{gray}(I)$; $G_4 \leftarrow \text{downsample}(G, 4)$
for $k = 1$ **to** 4 **do**
    $(x_k, y_k) \leftarrow \arg\max \rho_k$ on G_4, complete occurrences only; refine on G within ± 12 px
    mark region k missing if $\rho_k < 0.45$
end for
resolve overlaps $> 10\%$ by re-searching the lower-scoring region
for each found region k **do**
    $x \leftarrow \text{resize}(\text{rot90}(\text{crop}(I, k)), 256)$
    $A_k \leftarrow \mathcal{M}_k(x)$; $\sigma_k \leftarrow \text{mean}(A_k)$
end for
return Eq. (4.23), $\{\sigma_k\}$, $\{A_k\}$

4.2 System Integration

4.2.1 Pipeline per inspection

Each Station 1 inspection runs ingestion, inference, quality gate, audit logging, and overlay rendering in that order (Figure 4.2). Overlays are rendered on demand from stored detections, so the stored images stay clean and can be re-used for training.

4.2.2 Services, concurrency, and GPU memory

Each station is owned by a singleton service (`PipelineService`, `FillPipelineService`) that holds the engine, gate, logger, and renderer. Inference never runs on the server's event loop: asynchronous routes hand work to a thread pool. One TensorRT execution context is not thread-safe, so each service serialises inspections with an inference lock; loading and unloading

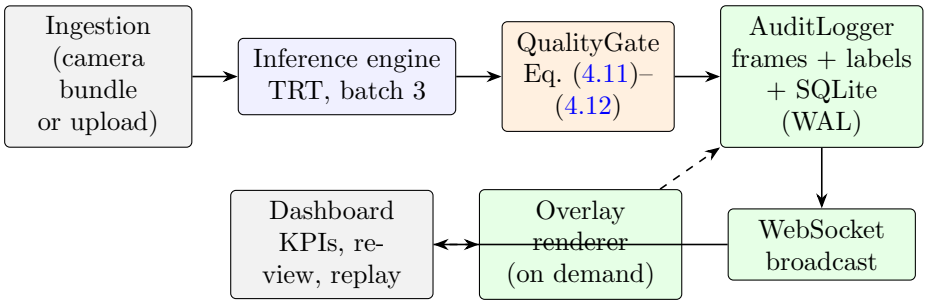


Figure 4.2: Integration flow for one Station 1 inspection. Solid arrows: per-inspection path; dashed: on-demand overlay requests from the dashboard.

an engine is guarded by a separate engine lock, and the two are always taken in the order inference lock, then engine lock, which rules out deadlock. Because a station switch can unload the engine at any moment, the code reads the engine reference once into a local variable before using it. On a switch the dashboard calls `POST /api/unload` and then `POST /api/fill/load` (or the reverse), so only one station's engines occupy GPU memory.

4.2.3 Audit trail

Station 1 writes one row per bottle to an `inspections` table (bottle identifier, timestamp, verdict, actuator flag, primary defect and confidence, flagged cameras, decision reason, latency, camera count, path of the stored bundle) and one row per defect to a `defect_events` table, with indices on identifier, verdict, time, and defect. Station 2 writes a `fill_inspections` table that adds the fill status, ratio, meniscus height, bottle height, and both detector confidences. Both databases run SQLite in write-ahead-log mode. Each bundle directory stores the raw frames and YOLO-format labels, so stored inspections can be exported directly as training data.

4.2.4 Operator dashboard (O5)

The dashboard is a React application served by nginx. A station context holds the active station and swaps the WebSocket stream, the API endpoints, and the page set. Table 4.3 lists what is implemented.

4.2.5 Engine export and deployment

Each PyTorch checkpoint is exported to ONNX and compiled to a TensorRT FP16 engine on the target GPU the first time the station loads. Algorithm 4 gives the procedure. The cap engine has a dynamic batch profile (optimum 3, maximum 4), so the three views of a bottle share one execution followed by NMS per view; the two fill engines have a static batch of 1 at 640 px. Engines are cached under a version tag; changing the tag or setting a force-recompile flag rebuilds them. If no compatible GPU is present, the service falls back to ONNX Runtime and then to PyTorch. All paths come from environment variables resolved against the repository root, so the same code runs natively and in Docker, where the variables point at bind-mounted host folders.

Table 4.3: Operator dashboard features

Feature	Implementation
Live view	Station 1: three-camera feed with per-view overlays and a conveyor filmstrip of recent bottles; Station 2: single feed with the fill-ratio ruler. Driven by the WebSocket stream.
KPI bar	Total inspected, compliance yield, rejection rate, and latest or average cycle latency. The throughput tile is currently estimated from the inspection count; the measured line rate comes from the stress test (Table 5.11).
Analytics	Defect-class breakdown (Station 1) and fill-ratio histogram (Station 2).
Audit page	Searchable, filterable, paginated history of all inspections with latency per record; CSV export (RFC 4180) for both stations.
Replay inspector	Re-opens any stored bottle with mask blending, per-view contours, decision reason, and navigation to adjacent bottles.
Review queue	`UNCERTAIN` products are listed for review; the operator corrects polygons or labels and exports the reviewed set as a YOLO dataset (active learning).
Settings	Per-station gate thresholds (τ_p, τ_d, τ_u; θ_u, θ_o) editable at run time.
Camera input	The ingestion layer has a threaded `VideoCapture` worker for USB and RTSP sources that keeps only the latest frame; the evaluation used uploads and the simulation feed, and live-camera operation was not tested.
Not implemented	Automated PDF shift report and a latency histogram view (listed as future work).

Algorithm 4 Engine load with versioned cache and fallback

Input: Checkpoint path p, version tag v, batch profile B
Output: Loaded predictor

if CUDA available **then**
  $e \leftarrow$ `<stem>_<v>.engine` next to p
  if e missing **or** force-recompile set **then**
    export $p \rightarrow$ ONNX $\rightarrow$ TensorRT FP16 with profile B *minutes on first run*
  end if
  load e; warm up (two single-image passes, plus a 3-view pass for the cap engine)
  if warm-up output shape $\neq$ expected batch **then**
    rebuild e once
  end if
else if ONNX Runtime installed **then**
  load `.onnx`
else
  load `.pt` with PyTorch
end if

4.2.6 Integration challenges

Table 4.4 records the main problems met during integration and how they were solved.

Table 4.4: Integration challenges and solutions

Challenge	Solution
TensorRT engine built with static batch 1 silently ran the three views one by one	Dynamic batch profile at export, batched warm-up check, automatic rebuild on shape mismatch.
Engines from one machine fail on another GPU	Compile on the target at first start; cache with version tag.
Two stations' engines in one 24 GB GPU	Lazy load of Station 2, explicit unload/load on switch, inference and engine locks.
Inference blocking the web server	Thread-pool execution of all inference calls.
Host paths differ between machines and Docker	Environment-driven paths resolved against the repository root; Compose maps host folders to fixed container paths.
Label models fed the wrong colour mode	Threshold files record the input mode and score statistic; the CLI and web app refuse mismatched settings.

4.3 Testing & Validation

Testing followed the layer structure. Each station has one test script per layer (ingestion, inference, gate, audit, visualiser, API), an in-process end-to-end test runs both stations and the station swap, a stress test drives the full Station 1 loop inside the container, and the label prototype has 30 model-free unit tests for localisation, explainability, and the web app. Model accuracy was validated on held-out test splits that were not used for training or model selection (Chapter 5). Table 4.5 lists representative test cases, including the failures.

Table 4.5: Representative test cases

Test case	Input	Expected	Actual
Cap gate, held-out test	354 test images, FP16 engine	FAR $\leq$ 3%, FRR $\leq$ 7%	0/153 and 0/201 (Passed)
FP16 vs FP32 accuracy	Test split	$\lvert\Delta\text{mAP}\rvert < 0.01$	$\leq$ 0.0013 overall (Passed)
Station swap	Cap→fill→cap in one process	Engines released and reloaded	Passed (in-process, CPU/ONNX)
Fill verdict agreement	41 meniscus test images	≥ 0.93	$34/35 = 0.971$ (Passed)
Fill, multi-bottle rack frames	4 rack frames	Graceful handling	No bottle found $\Rightarrow$ `INSPECTION_FAILED` (by design)
Closed loop at 300 bottles/min	500 unpaced cycles	p50 $\leq$ 200 ms	299 ms (Failed: storage-bound)
Paced run, 150 bottles/min	300 cycles	150 bottles/min	126.7 bottles/min (Failed)
Database integrity under load	800 cycles in total	No failed writes	0 failures (Passed)
Label localisation	120 NSC scans	All 4 ROIs found	480/480 (Passed)
Label overlap rule	120 NSC scans	No false re-assignment	Rule never fired (Passed)
Label ROI_4 precision	35 held-out crops	High precision	57.9% (Open issue)

Chapter 5

Results and Discussion

This chapter reports the measured results of every station, compares them with the Phase 1 approach and with the targets set in Chapter 1, and discusses what they do and do not show. Unless stated otherwise, accuracy figures are on held-out test splits that were not used for training, model selection, or threshold setting.

5.1 Experimental Setup

Training ran on an NVIDIA RTX A5000 (24 GB). All latency figures were measured inside the production Docker image on one workstation (RTX A5000, Intel Xeon W-2295 with 36 threads, CUDA 12.1 PyTorch build 2.4.1, TensorRT 10.16.1, Ultralytics 8.4.192, ONNX Runtime 1.31.1 with the CPU provider). Each GPU configuration was warmed up with 5 calls and timed over 100 calls; medians are reported. Station 1 accuracy is reported for both the trained FP32 checkpoint and the deployed FP16 engine. Label-prototype results come from the experiments on the `himalaya-label-detection` branch and are reported with the protocol under which each number was obtained.

5.2 Experimental Results

5.2.1 Cap dataset and artefacts (O1)

Table 5.1 gives the per-class counts found on disk. The Roboflow export notes 8,530 images, but the three splits contain 8,524. Every image shows one bottle neck with one annotated cap, captured at 2048×2048 (a handful at 2048×1536) against indoor backgrounds. The training split holds 2,486 source photographs, each exported in three augmented versions (rotation ±5°, brightness ±25%, blur up to 2 px, 0.5% salt-and-pepper noise); validation and test images are not augmented. The masks come from a single annotator, so inter-annotator agreement cannot be measured.

Table 5.1: Six-class cap dataset (one cap per image)

Class	Train	Valid	Test	Share of train
good_cap	4,242	405	201	56.9%
wet_cap	828	79	39	11.1%
open_cap	768	65	38	10.3%
damaged_cap	648	62	31	8.7%
no_cap	645	61	30	8.6%
misplaced_cap	327	40	15	4.4%
Total images	**7,458**	**712**	**354**	
Source photographs	2,486	712	354	

The cap is not a small object in this dataset: its median bounding box covers 9.2% of the test image (10th–90th percentile 6.0–14.0%), and after resizing to the 640-pixel network input

every instance is "large" by the COCO definition ($\geq 96^2$ pixels) [48]. What is small is the evidence inside the cap: droplets on wet caps, dents on damaged caps, and the thread gap of open caps. "Small-object-aware" in the primary objective therefore refers to preserving fine intra-object detail, which is what the distillation sites target, rather than detecting tiny objects.

Table 5.2 lists the exported artefacts of the cap model. The FP16 engine is about half the size of the FP32 ONNX graph, and Section 5.2.3 shows that it keeps the model's accuracy. As the benchmark for future model versions, a held-out test grouped by capture session has been identified, so that images of the same session never appear in both training and test.

Table 5.2: Cap model artefacts

Artefact	**Precision**	**Size (MB)**	**Use**
PyTorch checkpoint (`best.pt`)	FP32	45.2	Training, PyTorch fallback
ONNX graph	FP32	90.5	Portable runtime, TensorRT input
TensorRT engine (`best_v1.engine`)	FP16, dynamic batch	49.1	Deployment on the target GPU

5.2.2 Distillation outcome

Table 5.3 compares teacher and student, both evaluated in FP32 with the same Ultralytics validation on the same splits. Figure 5.1 shows the student's validation mAP during distillation training.

Table 5.3: Teacher versus distilled student (FP32, 640 px)

	Teacher YOLO11x-seg	**Student YOLO11m-seg**
Parameters (M)	62.06	22.34
GFLOPs @640	297.4	113.0
Checkpoint size (MB)	124.8	45.2
Inference per image (ms)*	19.0	7.8
Test split ($n = 354$)		
Box precision / recall	0.995 / 0.997	0.998 / **1.000**
Box / mask mAP@0.5	0.995 / 0.995	0.995 / 0.995
Box mAP@0.5:0.95	**0.944**	0.927
Mask mAP@0.5:0.95	0.912	**0.927**
Per-class mask mAP@0.5:0.95 (test)		
damaged / good	0.929 / 0.877	0.927 / **0.893**
misplaced / no cap	0.899 / 0.913	**0.941** / 0.915
open / wet	0.931 / 0.924	0.935 / **0.953**
Validation split ($n = 712$)		
Box / mask mAP@0.5:0.95	0.946 / 0.913	0.938 / **0.936**

*PyTorch FP32, batch 1, Ultralytics validation; teacher timed on its training GPU.

The student matches the teacher's mAP@0.5 of 0.995 on every class with 2.8× fewer parameters, 2.6× fewer FLOPs, and a 2.8× smaller checkpoint, and it runs 2.4× faster per image. At strict IoU the two models trade places. The student's *mask* mAP@0.5:0.95 is higher than the teacher's on both test (0.927 vs. 0.912) and validation (0.936 vs. 0.913), and higher for five of six classes, with damaged caps level (0.927 vs. 0.929). The gain is largest on the classes whose evidence is finest: wet (+0.029), misplaced (+0.042), and good (+0.016). The teacher keeps a lead in *box* mAP@0.5:0.95 (0.944 vs. 0.927). Station 1 decides from the cap's region and stores its mask, so the mask trade is the one the application favours. The student also has higher test

recall (1.000 vs. 0.997; the teacher misses one open cap) and makes no spurious background detection, where the teacher makes one.

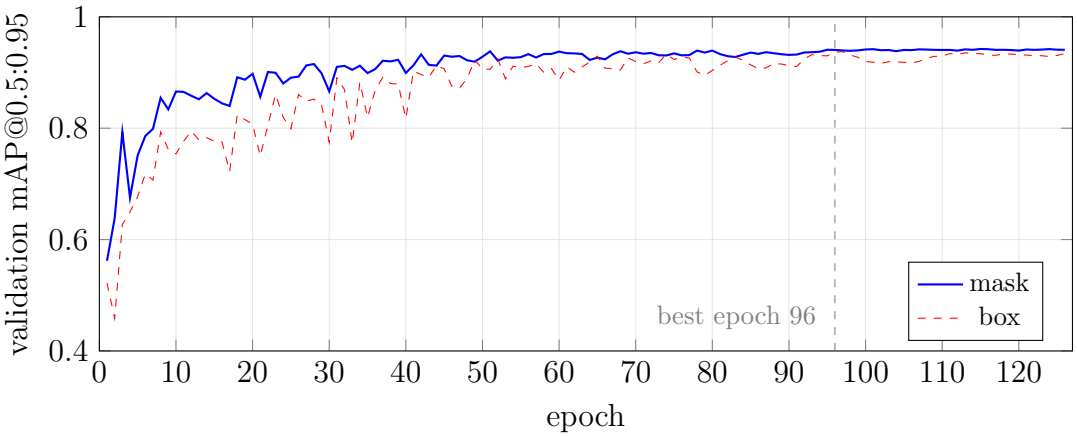


Figure 5.1: Validation mAP@0.5:0.95 of the YOLO11m-seg student during distillation training. Training stopped early at epoch 126 (patience 30).

Figure 5.2 shows the student's mask precision–recall curves and normalised confusion matrix on the validation split. Every class reaches AP@0.5 of 0.995; the only off-diagonal mass is a small confusion between misplaced and open caps.

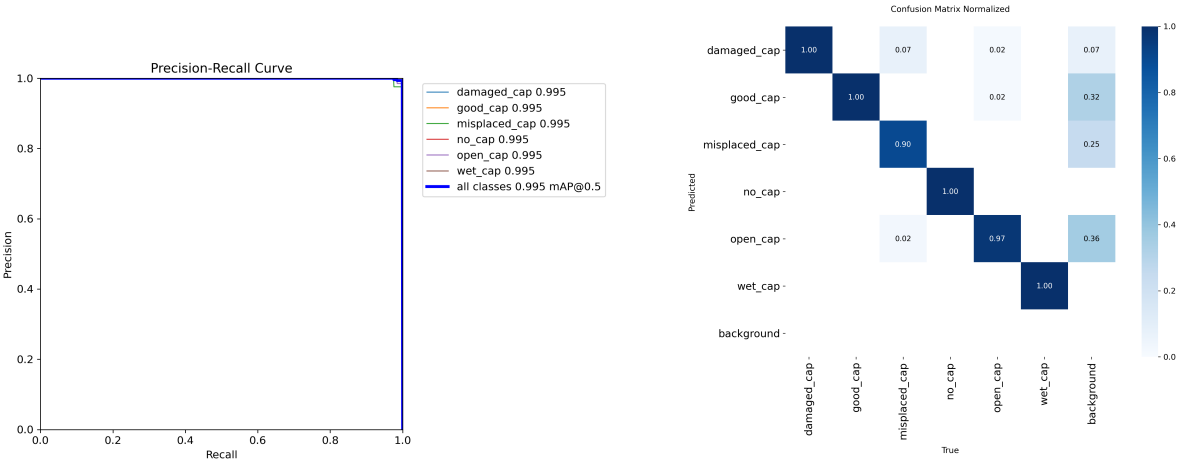


Figure 5.2: Student YOLO11m-seg on the validation split: (left) mask precision–recall curves per class; (right) normalised confusion matrix.

5.2.3 Cap segmentation: trained versus deployed model

Table 5.4 reports per-class segmentation metrics for the FP32 checkpoint and the deployed FP16 engine. Both reach AP@0.5 of 0.995 for every class. Converting to FP16 changes overall mAP@0.5:0.95 by at most 0.0013 and any single class by at most 0.006. The lowest strict-IoU score is for good-cap masks (0.893), where the boundary against the bottle neck is least distinct.

Taking the highest-scoring detection above the 0.35 floor as the image label, top-1 classification is correct for 354 of 354 test images and 710 of 712 validation images (macro-F1 0.993, Cohen's κ 0.996, MCC 0.996). Both validation errors are misplaced caps predicted as open caps; both are defect classes, so the bottle verdict is still `REJECT`.

Table 5.4: Station 1 segmentation on the held-out test split ($n = 354$)

	FP32 PyTorch		FP16 TensorRT	
Class (support)	**Mask $AP_{50:95}$**	**Box $AP_{50:95}$**	**Mask**	**Box**
damaged (31)	0.927	0.952	0.931	0.958
good (201)	0.893	0.904	0.894	0.903
misplaced (15)	0.941	0.930	0.942	0.930
no cap (30)	0.915	0.922	0.915	0.922
open (38)	0.935	0.928	0.933	0.929
wet (39)	0.953	0.929	0.953	0.930
All: $mAP_{50:95}$	0.927	0.927	0.928	0.929
All: P / R	0.998 / 1.000		0.998 / 1.000	
Valid: $mAP_{50:95}$	0.936	0.938	0.936	0.937

5.2.4 Product-level error and threshold analysis

Table 5.5 replays the production gate image by image with the deployed engine. Each image counts as one bottle seen by one view, because the dataset has no synchronised triplets. No defective cap is passed and no good cap is rejected. With zero observed events, the exact Clopper–Pearson [36] 95% upper bound on a rate with n trials is $1 - 0.025^{1/n}$, which gives 2.4% false accept and 1.8% false reject on the test split.

Table 5.5: Production gate replayed with the FP16 engine (defaults $\tau_p = 0.75$, $\tau_d = 0.45$, $\tau_u = 0.40$)

	Valid	**Test**
PASS / REJECT / UNCERTAIN / FAILED	405 / 307 / 0 / 0	201 / 153 / 0 / 0
False accept (defect → PASS)	0/307 ($\leq 1.2\%$)	0/153 ($\leq 2.4\%$)
False reject (good → REJECT)	0/405 ($\leq 0.9\%$)	0/201 ($\leq 1.8\%$)
Auto-decided accuracy	1.000	1.000

Parentheses: Clopper–Pearson 95% upper bound.

We swept $\tau_p \in [0.50, 0.95]$ and $\tau_d \in [0.25, 0.65]$ (Figure 5.3). No false accept or false reject occurs at any grid point. The defect threshold has no effect over its whole range, because the engine emits no wrong-polarity score above 0.01 on either split: there is no defect score on good caps and no good-cap score on defective caps. The pass threshold controls only the review load. Good-cap top scores lie in $[0.856, 0.921]$ on test (median 0.897), so review routing starts between $\tau_p = 0.85$ and 0.90; at 0.90, 34.5% of test bottles and 36.0% of validation bottles go to review. The default $\tau_p = 0.75$ leaves a margin of 0.106 below the lowest good-cap score. Defective caps score between 0.820 and 0.941 on test, 0.37 above τ_d. Accuracy is identical across bounding-box-area terciles; the only validation errors fall in the largest tercile.

5.2.5 Fill-level estimation (O2)

Table 5.6 reports both Station 2 detectors on their held-out test splits at 640 px. The meniscus detector localises the surface reliably at IoU 0.5 (AP 0.923) but loosely at strict IoU (AP@0.5:0.95 of 0.458). Because Eq. (4.16) uses only the box mid-line, this looseness affects the ratio far less than the strict-IoU score suggests. FP16 conversion leaves the meniscus detector unchanged but lowers the bottle detector's AP@0.5:0.95 from 0.578 to 0.481; the deployed bottle engine is built at 640 px although the detector was trained at 512 px, and building it at

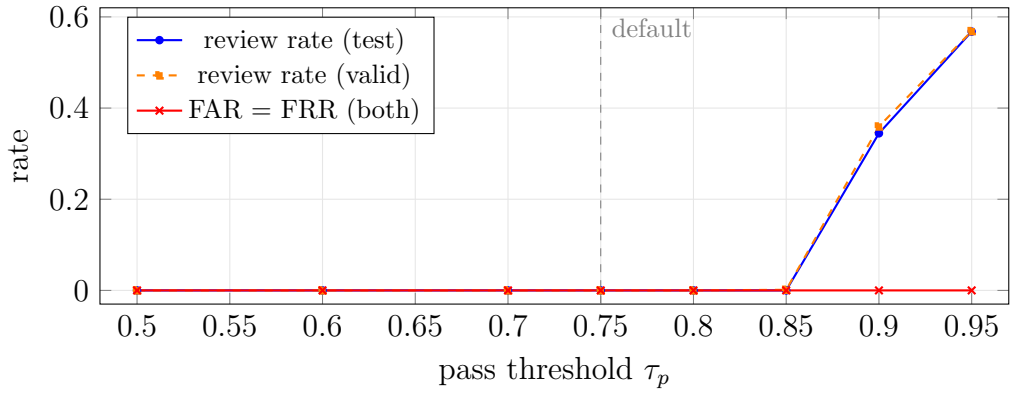


Figure 5.3: Gate threshold sweep with the FP16 engine ($\tau_d = 0.45$; every other τ_d in $[0.25, 0.65]$ gives identical curves).

its training resolution is the planned remedy.

Table 5.6: Station 2 detectors on held-out test splits (640 px)

Model (class, boxes)	**Runtime**	**P**	**R**	**AP$_{50}$**	**AP$_{50:95}$**
YOLOv8s (bottle, 356)	FP32	0.855	0.880	0.935	0.578
	TRT FP16	0.823	0.890	0.907	0.481
YOLOv8n (surface, 76)	FP32	0.904	0.867	0.923	0.458
	TRT FP16	0.883	0.882	0.921	0.459

To isolate the effect of meniscus localisation on the estimate, the deployed engines were run on the 41 surface-test images. For each image the ratio from the predicted surface was compared with the ratio from the annotated surface, using the same predicted bottle box. Table 5.7 gives the results. In 31 of 35 evaluable images the ratio differs by at most 0.02, the median difference is 1 pixel (0.002 in ratio), and the fill state agrees in 34 of 35 images. The single disagreement is an overfilled bottle (annotated ratio 0.902) that was passed as acceptable (0.708).

Table 5.7: Fill-ratio error from meniscus localisation (41 test images, TRT FP16)

No bottle detected (multi-bottle rack frames)	4
Bottle found, no surface associated	2
Evaluated	35
Median $\lvert\Delta R\rvert$ / MAE / 90th percentile / max	0.002 / 0.023 / 0.022 / 0.300
$\lvert\Delta R\rvert \le 0.02$ / ≤ 0.05	88.6% / 91.4%
Fill-state agreement (under / acceptable / over)	34/35 (97.1%)
Gate verdicts PASS / REJECT / UNCERTAIN	27 / 7 / 1

The large errors share one failure mode: when the true surface sits high in the neck, the detector fires on a horizontal structure lower on the body, such as a moulded shoulder rib, a glare band, or the edge of a gripping hand. The two missed surfaces are a tinted bottle held in a hand and a nearly full bottle. All four frames with no bottle detection come from the multi-bottle rack video, which the single-bottle design does not support; they are reported as `INSPECTION_FAILED` rather than guessed.

5.2.6 Label-defect prototype (O4)

Localisation. Template matching located all four regions in all 120 NSC scans (480 of 480), taking about 120 ms per scan on the CPU. The overlap rule never fired, so no region was re-assigned. Localisation therefore needed no labelled boxes and no GPU.

Colour versus grayscale models. The first EfficientAD models were trained on colour crops. On good crops they flagged large bright areas of the white label background, which produced many false alarms. Retraining on grayscale crops, replicated to three channels, made the models attend to print texture and intensity instead of colour. Table 5.8 compares the two for the crop-level evaluation.

Table 5.8: Label prototype: colour versus grayscale EfficientAD-Medium, crop-level

			Colour		Grayscale	
ROI	**Good**	**Bad**	**Recall**	**FP**	**Recall**	**FP**
ROI_1 (logo, name)	74	33	79%	11	93.9%	6
ROI_2 (ingredients)	77	14	86%	7	92.9%	6
ROI_3 (address, dates, barcode)	78	31	74%	8	93.5%	13
ROI_4 ("3 Way Care" graphic)	77	26	46%	high	92.3%	56

Final per-region results. ROI_4 kept a very high false-alarm count after the switch to grayscale, which pointed to noisy defect annotations rather than the architecture. Its defects were re-annotated, new crops were added, and the model was retrained for 300 epochs on 116 good and 60 defective crops. Table 5.9 gives the current per-region results. The protocols differ, and the table says so: ROI_1–3 were evaluated on the available crop sets, whose good crops overlap the training data; ROI_4 was evaluated on a 20% split (seed 20261029) that was separated after training but not used for training the threshold, so it is a post-training holdout rather than a fully independent estimate.

Table 5.9: Label prototype: grayscale EfficientAD-Medium per region

ROI	**Evaluation set**	**Acc.**	**Prec.**	**Recall**	**F1**	**TP/TN/FP/FN**
ROI_1	Crop set (74 good, 33 bad)	92.5%	83.8%	93.9%	88.6%	31/68/6/2
ROI_2	Crop set (77 good, 14 bad)	92.3%	68.4%	92.9%	78.8%	13/71/6/1
ROI_3	Crop set (78 good, 31 bad)	86.2%	69.0%	93.5%	79.5%	29/65/13/2
ROI_4 old	Holdout (15 good, 5 bad)	45.0%	28.6%	80.0%	42.1%	4/5/10/1
ROI_4 new	**Holdout (23 good, 12 bad)**	**74.3%**	**57.9%**	**91.7%**	**71.0%**	**11/15/8/1**

All four regions catch more than 91% of defective crops, slightly below the 95% recall that the threshold calibration targeted. Precision is the weak side: the models flag 6–13 of about 75 good crops in ROI_1–3, and 8 of 23 good crops in ROI_4.

Image-level status. The image-level verdict (Eq. (4.23)) has been tested only with provisional colour-mode thresholds fitted on 16 good and 16 defective scans that were also in the training data; a split-half check inside that subset caught 15 of 16 defective scans and flagged 3 of 16 good scans. Two earlier end-to-end runs illustrate why calibration matters: a run that

loaded an outdated localiser caught only 21.4% of defective scans, and a run with thresholds on the wrong score scale flagged all 120 scans (recall 1.0, accuracy 0.35). A full 120-scan run with the grayscale models and recalibrated thresholds, on held-out scans, has not yet been done.

Latency. On a 16-core CPU the ONNX anomaly models take about 7 s per crop, or about 25 s per scan. During training the EfficientAD-Medium models ran at roughly 50 ms per crop on the GPU, so GPU deployment is required before the prototype can approach line rate.

5.3 Performance Evaluation

5.3.1 Network latency (O3)

Table 5.10 isolates the network from the rest of the pipeline. TensorRT FP16 reduces the three-view cap network time from 21.0 ms (PyTorch FP32) to 7.4 ms, a 2.8× speed-up, and the single-image fill detectors drop from 8.8 and 8.3 ms to 1.3 and 1.1 ms. ONNX Runtime on the CPU needs 510 ms for the cap batch, which is too slow for the line. Measured end to end through the predictor, three 2048-pixel views take 20.1 ms, of which about 13 ms is host-side resizing and mask post-processing that grows with batch size and is not accelerated by the engine.

Table 5.10: Per-call latency (ms, median of 100) inside the production container

Model / input	Runtime	Network	Pre+post	Wall
Cap seg., 3 views 2048^2	PyTorch FP32	20.97	12.47	33.79
	TRT FP16	7.44	12.35	20.12
	ORT CPU FP32*	509.89	16.68	526.43
Cap seg., 3 views 640^2	TRT FP16	7.39	7.56	15.25
Cap seg., 1 view 2048^2	TRT FP16	3.90	4.38	8.59
Bottle detector	PyTorch FP32	8.77	2.11	11.21
	TRT FP16	1.26	2.07	3.60
Meniscus detector	PyTorch FP32	8.34	2.52	11.15
	TRT FP16	1.13	2.33	3.76
Fill engine (both detectors + Eq. (4.16))	TRT FP16	p50 7.34 / p95 10.23		
Fill gate (Alg. 2)	CPU	p50 0.028 / p99 0.059		

*20 calls. Pre+post: Ultralytics pre-processing and post-processing.

5.3.2 Closed-loop stress test

The Station 1 stress test drives the full production path inside the container for 500 cycles: TensorRT inference, extraction of mask polygons at native resolution, the gate, a synchronous SQLite (WAL) transaction that stores the three raw frames with labels, and overlay rendering. It cycles through the 354 test images, grouped three per bottle. Table 5.11 gives the breakdown. The median cycle is 299 ms, or 163 bottles per minute unpaced. In a second run paced at 150 bottles per minute for 300 cycles, the line sustained 126.7 bottles per minute with a 99th-percentile cycle of 448 ms; the JPEG decode of each 2048-pixel triplet falls outside the timed cycle, which is why the paced run fell short. No database write failed in either run.

The dominant cost is not the network. Storing three 2048-pixel frames synchronously takes 176 ms, and most of the 67 ms inference stage is mask-polygon extraction at native resolution.

Table 5.11: Station 1 closed-loop stress test, 500 unpaced cycles (ms)

Stage	p50	p95	p99
Inference (3 views, incl. mask polygons)	67.15	122.17	124.69
Quality gate	0.12	0.15	0.18
SQLite WAL audit (3 raw frames)	176.48	251.87	276.67
Overlay rendering (3 views)	41.59	43.21	43.91
Total closed-loop cycle	**299.18**	**392.05**	**428.71**

Station 2's service path (both engines, gate, audit, and overlay) takes 62.8 ms at the median and 111.5 ms at the 99th percentile over 500 frames, because it stores one frame instead of three.

5.4 Validation

Validation relied on hold-out sets at every level. The cap models were selected on the 712-image validation split and reported on the 354-image test split, which was not used for selection or threshold setting; agreement between the two splits (Table 5.4) shows no sign of over-fitting to the selection set. Class-sensitive statistics were used because the classes are imbalanced: per-class AP, macro-F1 0.993, Cohen's κ 0.996, and MCC 0.996 on validation, and exact binomial bounds wherever an error count is zero (Table 5.5). The fill estimate was validated against annotated surfaces on a held-out split, isolating meniscus error from bottle-box error. During system integration the deployed cap engine was also run on photographs of our own bottles, which the model had never seen in training or evaluation, and its verdicts on these were satisfactory. For the label prototype, the strongest validation available is the ROI_4 post-training holdout; the other regions still need an independently reserved test set.

5.5 Comparative Analysis

5.5.1 Why a segmentation detector replaced the Phase 1 classifier

Phase 1 inspected caps in two stages: a YOLO detector found bottles and caps, and a MobileNetV3-Small classifier labelled the cap crop as good, defective, or missing. Before redesigning Station 1, five whole-image cap classifiers were benchmarked on a separate four-class dataset (7,540 images; 308-image test split). Table 5.12 gives the results.

Table 5.12: Whole-image cap classifiers (separate 4-class dataset, $N_{\text{test}} = 308$). GPU: PyTorch, batch 1; CPU: ONNX Runtime INT8

Model	Acc. %	mF1 %	Damaged F1	Params (M)	GPU ms	CPU ms
MobileNetV3-L [49]	96.43	94.75	0.857	4.21	9.36	4.77
EfficientNetV2-S [50]	95.78	93.96	0.831	20.18	38.86	20.82
ShuffleNetV2 [51]	94.81	92.84	0.794	1.26	8.55	3.48
ResNet-50 [52]	94.81	92.30	0.788	23.52	13.14	11.19
DeiT-Tiny [53]	94.81	92.69	0.788	5.53	12.64	7.07

These numbers come from a different dataset and label set, so they are not directly comparable with Station 1. They informed the redesign in two ways. First, even the best classifier stayed below 0.86 F1 on damaged caps, the class whose evidence is finest. Second, a classifier

returns no location, whereas the gate and the audit record need the cap's region (mask area, polygon, and per-view evidence). Table 5.13 sets the two Station 1 designs side by side.

Table 5.13: Phase 1 versus Phase 2 cap inspection (different datasets and label sets; indicative only)

	Phase 1: YOLO detector + MobileNetV3-Small	**Phase 2: distilled YOLO11m-seg**
Classes	3 (good, defective, missing)	6 (good, damaged, misplaced, missing, open, wet)
Output	Class label per cap crop	Class, box, and mask per cap in one pass
Detector accuracy	P 0.869, R 0.973, mAP@0.5 0.958, mAP@0.5:0.95 0.786	P 0.998, R 1.000, mAP@0.5 0.995, mask mAP@0.5:0.95 0.928 (FP16, test)
Cap classification	86.5% accuracy (P = R = 0.870)	100% top-1 on test (354/354); 99.7% on validation
Product-level errors	Not measured	FAR 0/153, FRR 0/201
Evaluation split	Final training epoch, validation	Untouched test split, FP32 and deployed FP16
Latency	466–678 ms per uploaded image (dashboard, development machine)	7.4 ms network, 20.1 ms predictor for three views

5.5.2 Comparison with the literature

Direct numerical comparison with Table 2.1 is not meaningful because every study uses a different product and dataset. Qualitatively, Phase 2 differs in what it reports: product-level false-accept and false-reject rates with exact bounds, as Banús *et al.* [1] also do; a threshold sweep with an explicit margin, which none of the reviewed studies report; a measured fill ratio instead of a fill class [6]; and per-stage and closed-loop latency on the deployed system.

5.6 Model Optimization & Innovation

Optimisation steps and their effect.

- **Feature distillation:** 62.1 M → 22.3 M parameters (2.8×), 297.4 → 113.0 GFLOPs, 19.0 → 7.8 ms per image in FP32, with equal mAP@0.5 and +0.015 mask mAP@0.5:0.95 on test.
- **TensorRT FP16:** cap network 21.0 → 7.4 ms for three views (2.8×); fill detectors 8.8 → 1.3 ms and 8.3 → 1.1 ms; accuracy change ≤ 0.0013 mAP for the cap model.
- **Dynamic batching:** the three views share one execution instead of three.
- **Grayscale label crops:** recall on ROI_1–3 raised from 74–86% to 93–94%.

Innovative aspects. The contributions specific to this project are: (i) feature distillation at the three YOLO11 blocks that carry texture, multi-scale context, and attention, with L_2-normalised alignment that reduces to a cosine distance (Eq. (4.8)); (ii) an asymmetric multi-view gate whose thresholds are analysed for margin and review load; (iii) a closed-form fill ratio with a derived sensitivity and a confidence-gated review band; and (iv) a label prototype that combines training-free localisation with good-only anomaly models and counterfactual explanations.

Failed or abandoned attempts. PatchCore was abandoned for the label task because its memory bank made inference too slow. A YOLO11m localiser with SAHI tiling was trained for the label regions but replaced by template matching, which is deterministic and needs no boxes. Colour-trained label models were replaced by grayscale ones. The first cap engine used a static batch of 1 and silently ran the three views sequentially until it was rebuilt with a dynamic profile. The deployed bottle engine at 640 px loses 0.097 AP@0.5:0.95 against FP32; this is not yet fixed.

5.7 Summary Against the Stated Targets

Table 5.14 compares every measured quantity with its target, including the misses. The false-accept and false-reject limits follow from the recall (≥ 0.97) and precision (≥ 0.93) targets, and the 200 ms cycle from a line rate of 300 bottles per minute.

Table 5.14: All targets and outcomes (test split unless noted)

Quantity	Target	Measured	Met
Cap image-level accuracy	≥ 0.93	1.000	yes
Cap false accept / false reject	$\leq 3\%$ / $\leq 7\%$	0% / 0%	yes
Cap mask mAP@0.5:0.95 (FP16)	≥ 0.75	0.928	yes
Cap precision / recall (FP16)	≥ 0.93 / 0.97	0.998 / 1.000	yes
Bottle det. AP@0.5 / 0.5:0.95 (FP16)	≥ 0.88 / 0.75	0.907 / 0.481	yes / no
Meniscus P / R (FP16)	≥ 0.93 / 0.97	0.883 / 0.882	no / no
Meniscus AP@0.5 / 0.5:0.95 (FP16)	≥ 0.88 / 0.75	0.921 / 0.459	yes / no
Fill verdict agreement	≥ 0.93	0.971 (34/35)	yes
GPU inference, 3 views (network)	< 100 ms	7.4 ms	yes
GPU inference stage, p95	< 100 ms	122 ms	no
Closed-loop cycle, p50 (300 bottles/min)	≤ 200 ms	299 ms	no
Fill service cycle, p50 / p99	< 100 ms	62.8 / 111.5 ms	p50 only
CPU inference (ORT, 3 views)	< 200 ms	510 ms	no
Label defect recall per ROI (crop level)	≥ 0.95	0.917–0.939	no
Label ROI_4 precision (holdout)	—	0.579	open

5.8 Discussion and Limitations

Cap station. The distilled student is accurate, stable under FP16, and well separated by its gate thresholds. The sweep also gives a monitoring rule for production: if the good-cap score distribution drifts down by more than about 0.1, for example under new lighting or bottle types, the default τ_p will begin routing good bottles to review before any defect is passed. The session-grouped held-out test identified under O1 is the next benchmark and will make results comparable across future model versions. A YOLO11m-seg trained without distillation would isolate the contribution of each distilled layer.

Fill station. Measured against the annotated surface, the geometric rule is accurate wherever the meniscus is found; its errors are detection errors, not arithmetic errors. The rule assumes one upright bottle per frame and a visible meniscus inside the bottle box. The analysis isolates meniscus error only, because the surface dataset has no bottle annotations, so bottle-box error

is bounded analytically (Eq. (4.19)) rather than measured. No physical fill-volume ground truth was available.

Label prototype: unresolved ROI limitations.

- **False alarms.** ROI_4 precision is 57.9% on its holdout, and ROI_3 flags 13 of 78 good crops; the line would discard good tubes at an unacceptable rate if deployed today.
- **Image-level labels.** Defective scans are labelled per image, not per region: only 14 of 42 defective scans have a defect in ROI_2 and 26 in ROI_4, which dilutes threshold calibration.
- **No independent test set.** Thresholds for ROI_1–3 were fitted on data that overlaps training, and the image-level thresholds are provisional.
- **Controlled acquisition only.** All 120 scans are flat, evenly lit, unwrapped images. Specular reflections on the glossy tube, camera jitter, motion blur, and uncontrolled shop-floor lighting are not represented, and template matching with a fixed scale has not been tested under them; a change in scan resolution or label artwork requires new templates.
- **Latency.** About 25 s per scan on the CPU; GPU deployment of the anomaly models is still to be done.

Throughput. Real-time capability depends on the input/output around the network more than on the network. Writing the audit record asynchronously and down-sampling the stored frames would remove most of the 299 ms cycle; neither is implemented in the measured system.

Chapter 6

Conclusion and Future Discussion

6.1 Conclusion

Phase 2 turned the broad Phase 1 proposal into inspection stations that were built, measured, and deployed. For the primary objective, the two-stage Phase 1 cap pipeline (86.5% classifier accuracy, no location, no measured error rates) was replaced by a YOLO11m segmentation student distilled from a YOLO11x-seg teacher. With 2.8 times fewer parameters the student matched the teacher's mAP@0.5 of 0.995 and segmented caps more precisely at strict IoU (mask mAP@0.5:0.95 of 0.927 against 0.912). Behind an asymmetric multi-view gate, the deployed FP16 engine made no false accepts among 153 defective caps and no false rejects among 201 good caps on the untouched test split, and the threshold sweep showed a margin of 0.106 between the default pass threshold and the lowest good-cap score (Section 5.2.4).

The other objectives were met to different degrees. The dataset and artefacts were audited, the FP16 engine was shown to preserve accuracy, and a session-grouped test was identified as the next benchmark (O1). The geometric fill estimator reached a median ratio error of 0.002 from meniscus localisation and 97.1% verdict agreement, with every failure traced to a specific detection mode (O2). TensorRT reduced network time for three views to 7.4 ms, but the full closed loop ran at 299 ms because audit storage is synchronous, so the 200 ms target was missed (O3). The label prototype located all four print regions on all 120 scans and caught 91–94% of defective crops per region, but ROI_4 precision of 57.9% and the lack of an independent test set keep it at prototype stage (O4). The dashboard, review queue, replay, and SQLite audit trail run in Docker; the automated PDF shift report was not built (O5).

The broader lesson is that measurable inspection quality comes from three things working together: a model that keeps fine detail at deployable cost, a decision rule whose thresholds are analysed rather than assumed, and a deployment that stores the evidence for every verdict.

6.2 Key Learnings

- **Report what the plant needs.** Moving from validation accuracy to product-level false-accept and false-reject rates with exact confidence bounds changed how results were read: zero errors on 153 defects still means the true rate may be up to 2.4%.

- **Distil where the evidence lives.** Feature alignment at the texture, context, and attention blocks gave a student that beat its teacher on mask quality. L_2 normalisation, which turns the loss into a cosine distance, was what kept the background from dominating.

- **Measure the whole loop.** The network took 7.4 ms, yet the cycle took 299 ms. Profiling every stage showed that storage, not inference, was the bottleneck.

- **Prefer measurement to classification where a quantity exists.** A fill ratio with configurable limits and a derived sensitivity is easier to audit and adapt than a fill-level classifier with similar accuracy.
- **Calibration is part of the model.** In the label prototype, wrong weights or thresholds on the wrong score scale moved image-level recall from 21% to 100% with no change to the networks.
- **Engineering practice.** Version-tagged engines, environment-driven paths, lock ordering for GPU access, and Docker deployment made the system reproducible across machines; Git history made it possible to reconstruct the timeline in Figure 3.5.

6.3 Research Publication Status

The Phase 1 work was submitted to CSITSS 2026 (Paper 1204) and was not accepted; the reviews were received in August 2026. Every reviewer comment was mapped to a concrete change, and the manuscript was rewritten around the two measured stations as "VisionFood QAI: Distilled Bottle-Cap Defect Segmentation with Threshold Analysis and Geometric Fill-Level Estimation, Deployed with TensorRT" in the IEEE conference format (7 pages, 32 references), with a point-by-point response to the reviewers and the evaluation scripts and outputs as supplementary material. The revised manuscript is ready for resubmission; a similarity report from an approved tool will accompany it.

6.4 Future Work

In order of priority:

1. **Asynchronous audit storage.** Move the database write and frame storage off the inspection path and store down-sampled frames; the stress test shows this is the largest single saving toward the 200 ms cycle (Table 5.11).
2. **Session-grouped benchmark and ablation.** Evaluate the cap student on the session-grouped held-out test (O1), and train a YOLO11m-seg without distillation to quantify each distilled layer's contribution.
3. **Bottle engine at training resolution.** Rebuild the bottle-detector engine at 512 px to recover the 0.097 AP@0.5:0.95 lost at 640 px.
4. **Label prototype to deployment.** Reserve an untouched set of good and defective scans, label defects per region, recalibrate the grayscale thresholds, reduce ROI_4 false alarms, run the anomaly models as GPU engines, and test under glossy-surface reflections, camera jitter, and shop-floor lighting.
5. **Fill-level robustness.** Add bottle annotations to the surface data to measure bottle-box error, collect volumetric ground truth, and suppress shoulder-rib and glare false meniscus detections.

6. **Live cameras and reports.** Validate the USB/RTSP ingestion path on a moving conveyor with synchronised triggers, and add the automated shift report and a latency histogram to the dashboard.

6.5 SDGs Achieved

Table 6.1 links the project to three Sustainable Development Goals and names who benefits.

Table 6.1: SDG alignment, beneficiaries, and evidence

SDG (target)	**Link to this project and evidence**	**Beneficiaries**
SDG 9: Industry, Innovation and Infrastructure (9.4, 9.b)	Automated inspection that runs on one workstation GPU: a 22 M-parameter distilled model and FP16 engines bring the network time for three views to 7.4 ms, without a cloud service or a multi-GPU server. Public datasets and open-source tools keep the cost of adoption low.	Small and mid-size bottlers and cosmetic manufacturers, including MSMEs, that cannot afford proprietary inspection systems.
SDG 12: Responsible Consumption and Production (12.3, 12.5)	The asymmetric gate sends borderline products to review instead of the reject bin, and the measured false-reject bound (1.8% on test) limits good product thrown away. The fill ratio flags overfill as well as underfill, so excess product is caught at the filler. Stored audit images are re-used for training through the review queue, reducing new data collection.	Producers (less scrap and give-away of product) and the environment (less plastic and product waste).
SDG 3: Good Health and Well-being	Open, missing, misplaced, or wet caps compromise the seal; the cap station passed none of 153 defective caps on test. The label prototype checks the region that carries manufacturing and expiry dates, regulatory numbers, and the barcode (ROI_3), where a missing print can mislead a consumer.	Consumers, and quality-assurance operators who review only the uncertain cases instead of every product.

Environmental note. Distillation cut the operations per inspection by 2.6 times (297.4 to 113.0 GFLOPs), and FP16 execution further shortens GPU time per bottle. Power draw was not measured, so no energy saving is claimed.

Appendices

The appendices hold supporting material referred to in the main chapters: source excerpts of the decision logic (Appendix A), the REST and WebSocket interface of the deployed system (Appendix B), and the label-prototype calibration record (Appendix C).

Chapter A

Appendix A: Source Excerpts

Listing A.1 is the per-view rule of the Station 1 quality gate (Eq. (4.11)), taken from `pipeline/decision/gate.py`; the result objects are abbreviated. Listing A.2 is the fill-ratio computation of Eq. (4.14)–(4.16), taken from `pipeline/fill_level/inference/engine.py`.

```
def evaluate_view(self, camera_result):
    detections = camera_result.detections
    # 1. No detection above the 0.35 floor
    if not detections:
        return ViewVerdict(verdict=INSPECTION_FAILED, ...)

    # 2. Any defect class at or above the defect threshold
    defect_candidates = [
        d for d in detections
        if self.config.is_defect(d.class_name)
        and d.confidence >= self.config.defect_conf_threshold]
    if defect_candidates:
        top_defect = max(defect_candidates, key=lambda d: d.confidence)
        return ViewVerdict(verdict=REJECT, defect_name=top_defect.class_name
    , ...)

    # 3. Certified pass needs good_cap at or above the pass threshold
    top = max(detections, key=lambda d: d.confidence)
    if top.class_name == self.config.pass_class:
        if top.confidence >= self.config.pass_conf_threshold:
            return ViewVerdict(verdict=PASS, ...)
        elif top.confidence >= self.config.uncertain_floor:
            return ViewVerdict(verdict=UNCERTAIN, ...)   # borderline good
    cap

    # 4. Anything else goes to review, never to an automatic pass
    return ViewVerdict(verdict=UNCERTAIN, ...)
```

Listing A.1: Per-view quality gate (abbreviated)

```
@classmethod
def compute_fill_ratio(cls, bottle_bbox, water_bbox):
    bx1, by1, bx2, by2 = bottle_bbox
    bh = by2 - by1

    by1_corr = by1 + int(bh * cls.BOTTLE_TOP_TRIM)   # 0.03
    by2_corr = by2 - int(bh * cls.BOTTLE_BOT_TRIM)   # 0.02
    bottle_h_corr = max(by2_corr - by1_corr, 1)

```

```
    # Meniscus mid-line, clamped inside the trimmed bottle
    water_y = (water_bbox[1] + water_bbox[3]) // 2
    water_y = max(by1_corr, min(by2_corr, water_y))

    fill_from_bottom = by2_corr - water_y
    return round(float(fill_from_bottom / bottle_h_corr), 3)
```

Listing A.2: Fill-ratio computation

Chapter B

Appendix B: Interface of the Deployed System

Table B.1: REST and WebSocket endpoints

Endpoint	**Purpose**
Station 1 (cap)	
`GET /api/health`	Engine and runtime status
`POST /api/load`, `POST /api/unload`	Load or release the cap engine (station switch)
`POST /api/inspect/image`, `POST /api/simulate`	Inspect uploaded views or the next simulated bottle
`GET /api/stats`, `GET /api/defects`	KPI summary and defect history
`GET /api/audit/export`	Audit export as CSV
`GET /api/bottles/{id}`, `.../adjacent`, `.../image/{camera}`	Replay of a stored bottle
`GET|POST /api/config`	Read or change gate thresholds at run time
`GET /api/active-learning/candidates`, `POST .../bottle/{id}/save`, `GET .../export-dataset`	Review queue and YOLO dataset export
`WS /ws/stream`	Live inspection stream
Station 2 (fill level)	
`GET /api/fill/health`, `POST /api/fill/load|unload`	Status (including thresholds and VRAM state), lazy load and release
`POST /api/fill/inspect`, `POST /api/fill/simulate`	Inspect an uploaded or simulated frame
`GET /api/fill/stats`, `GET /api/fill/ratio-histogram`	KPIs and fill-ratio distribution
`GET /api/fill/defects`, `GET /api/fill/bottle/{id}`, `GET /api/fill/export-csv`	History, replay, CSV export
`GET|POST /api/fill/config`	Read or change θ_u, θ_o and the review band at run time
`GET /api/fill/active-learning/*`	Review queue and dataset export
`WS /ws/fill/stream`	Live inspection stream

Chapter C

Appendix C: Label Prototype Calibration Record

Table C.1: Calibrated thresholds of the label prototype

Region	**Colour profile (provisional)**	**Grayscale profile (new ROI_4)**
ROI_1	−0.0229	−0.0395
ROI_2	0.2282	−0.0414
ROI_3	0.0647	−0.0188
ROI_4	0.2558	−0.0317
Score statistic	mean of anomaly map	mean of anomaly map
Selection	image-level max F1, 16 good + 16 bad scans	ROI_4: 95% recall target on calibration split
Status	provisional; fitted on training scans	ROI_4 to be confirmed on a reserved holdout

Thresholds are specific to the input mode (colour or grayscale) and the score statistic they were fitted for; the threshold files record both, and the software refuses a mismatched combination. Negative values arise because the normalised anomaly maps are centred near zero on good crops.

References

[1] N. Banús, I. Boada, P. Xiberta, P. Toldrà, and N. Bustins, "Deep learning for the quality control of thermoforming food packages," *Scientific Reports*, vol. 11, 2021.

[2] O. Farhangi, E. Sheidaee, and A. Kisalaei, "Machine vision for detecting defects in liquid bottles: An industrial application for food and packaging sector," *Cluster Computing and Data Science*, 2024.

[3] G. Liu, S. Zhang, L. Wang, X. Li, and G. Li, "Research on mechanical automatic food packaging defect detection model based on improved YOLOv5 algorithm," *PLOS ONE*, vol. 20, 2025.

[4] K. G. Liakos, V. Athanasiadis, E. Bozinou, and S. I. Lalas, "Machine learning for quality control in the food industry: A review," *Foods*, vol. 14, 2025.

[5] R. R. Subramanian, M. Dhamini, M. Srija, M. Vaishnavi, and M. Vidyadhari, "Revolutionizing manufacturing quality control with optimized ResNet50: A deep learning approach for real-time defect detection," in *Proc. Int. Conf. Computational Robotics, Testing and Engineering Evaluation (ICCRTEE)*, 2025.

[6] B. Chatchapol, W. Autanan, and W. Anan, "Liquid filling quality control system for beverage industry using image processing and CNN," in *Proc. 24th Int. Conf. Control, Automation and Systems (ICCAS)*, 2024, pp. 1–6.

[7] X.-T. Nguyen, T.-T. Mac, Q.-D. Nguyen, and H.-A. Bui, "An industrial system for inspecting product quality based on machine vision and deep learning," *Vietnam Journal of Computer Science*, vol. 12, no. 2, pp. 193–208, 2025.

[8] K. K. S, M. C. G, and R. T. V, "Deep learning based product defect detection for sustainable smart manufacturing," in *Proc. 5th IEEE Global Conf. Advancement in Technology (GCAT)*, 2024, pp. 1–6.

[9] F. Xiong, N. Kühl, and M. Stauder, "Designing a computer-vision-based artifact for automated quality control: A case study in the food industry," *Annals of Operations Research*, 2024.

[10] P. Krishnan, U. S. Ebenezar, R. Ranitha, N. Purushotham, and T. S. Balakrishnan, "AI-driven intelligent IoT systems for real-time food quality monitoring and analysis," in *Proc. Int. Conf. Trends in Quantum Computing and Emerging Business Technologies*, 2024, pp. 1–5.

[11] Z. Cao, Z. Wu, and J. Gray, "RFID tag-integrated multi-sensors with AIoT cloud platform for food quality analysis," *Electronics*, vol. 15, no. 1, 2025.

[12] T. Du, "Online measurement and detection technology and its optimized application in intelligent manufacturing environment," *Procedia Computer Science*, vol. 262, pp. 83–91, 2025.

[13] A. M. Truong and H. Q. Luong, "A non-destructive, autoencoder-based approach to detecting defects and contamination in reusable food packaging," *Current Research in Food Science*, vol. 8, p. 100758, 2024.

[14] A. G. Ribeiro, L. Vilaça, C. Costa, T. S. da Costa, and P. M. Carvalho, "Automatic visual inspection for industrial application," *Journal of Imaging*, vol. 11, 2025.

[15] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[16] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[17] G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[18] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, "YOLACT: Real-time instance segmentation," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019, pp. 9157–9166.

[19] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2961–2969.

[20] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015.

[21] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "FitNets: Hints for thin deep nets," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[22] S. Zagoruyko and N. Komodakis, "Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.

[23] G. Chen, W. Choi, X. Yu, T. Han, and M. Chandraker, "Learning efficient object detection models with knowledge distillation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 742–751.

[24] T. Wang, L. Yuan, X. Zhang, and J. Feng, "Distilling object detectors with fine-grained feature imitation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4933–4942.

[25] P. Micikevicius *et al.*, "Mixed precision training," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2018.

[26] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[27] NVIDIA Corporation, "NVIDIA TensorRT developer guide," 2025. [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/

[28] ONNX Runtime developers, "ONNX Runtime," 2021. [Online]. Available: https://onnxruntime.ai

[29] M. Buda, A. Maki, and M. A. Mazurowski, "A systematic study of the class imbalance problem in convolutional neural networks," *Neural Networks*, vol. 106, pp. 249–259, 2018.

[30] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-balanced loss based on effective number of samples," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9268–9277.

[31] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[32] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2018.

[33] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix: Regularization strategy to train strong classifiers with localizable features," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019, pp. 6023–6032.

[34] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[35] D. Chicco and G. Jurman, "The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, art. no. 6, 2020.

[36] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.

[37] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD – A comprehensive real-world dataset for unsupervised anomaly detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9592–9600.

[38] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards total recall in industrial anomaly detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 14318–14328.

[39] K. Batzner, L. Heckler, and R. König, "EfficientAD: Accurate visual anomaly detection at millisecond-level latencies," in *Proc. IEEE/CVF Winter Conf. Applications of Computer Vision (WACV)*, 2024, pp. 128–138.

[40] S. Akcay, D. Ameln, A. Vaidya, B. Lakshmanan, N. Ahuja, and U. Genc, "Anomalib: A deep learning library for anomaly detection," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2022, pp. 1706–1710.

[41] J. P. Lewis, "Fast normalized cross-correlation," in *Proc. Vision Interface*, 1995, pp. 120–123.

[42] R. Brunelli, *Template Matching Techniques in Computer Vision: Theory and Practice*. Chichester, U.K.: Wiley, 2009.

[43] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing aided hyper inference and fine-tuning for small object detection," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2022, pp. 966–970.

[44] T. Karras, M. Aittala, J. Hellsten, S. Laine, J. Lehtinen, and T. Aila, "Training generative adversarial networks with limited data," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[45] "bottle_cap_sdp dataset, version 7," Roboflow Universe, 2024, CC BY 4.0. [Online]. Available: https://universe.roboflow.com/bottlecapproject-dyaxy/bottle_cap_sdp

[46] "fill-level dataset, version 1," Roboflow Universe (majid-alikhani), CC BY 4.0. [Online]. Available: https://universe.roboflow.com/majid-alikhani/fill-level

[47] "bottle-liquid-level dataset, version 2," Roboflow Universe (aisut), CC BY 4.0. [Online]. Available: https://universe.roboflow.com/aisut/bottle-liquid-level

[48] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755.

[49] A. Howard *et al.*, "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019, pp. 1314–1324.

[50] M. Tan and Q. Le, "EfficientNetV2: Smaller models and faster training," in *Proc. Int. Conf. Machine Learning (ICML)*, 2021, pp. 10096–10106.

[51] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical guidelines for efficient CNN architecture design," in *Proc. European Conf. Computer Vision (ECCV)*, 2018, pp. 116–131.

[52] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[53] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *Proc. Int. Conf. Machine Learning (ICML)*, 2021, pp. 10347–10357.